

Pontificia Universidad Javeriana
Facultad de Ingeniería — Ingeniería de Sistemas

Software Architecture Document
SAD

GammaLab v2.0

Proyecto	Plataforma de análisis y visualización de señales neurofisiológicas Gamma Lab
Documento	Software Architecture Document — SAD
Versión	1.0
Estado	En elaboración
Fecha	27 de agosto de 2026

Autores

Juan Felipe Romero Ortiz — juanromero@javeriana.edu.co
Laura Juliana Garzón Arias — garzonlj@javeriana.edu.co
Xiomara Herrera Suárez — x_herrera@javeriana.edu.co
Sebastián Almanza Galvis — s.almanzag@javeriana.edu.co

Nota de uso. Documento académico/interno del equipo de proyecto. No distribuir sin autorización.

Trazabilidad. Este SAD define la arquitectura del sistema y traza su relación con el SRS de GammaLab v2.0.

Índice

1. Introducción	6
1.1. Propósito	6
1.2. Alcance	6
1.3. Definiciones y acrónimos	6
1.4. Audiencia	7
1.5. Referencias relacionadas	7
1.6. Visión General	7
1.7. Núcleo con pocas responsabilidades y contratos débiles	7
1.8. Estado y memoria	8
1.9. Concurrencia y procesos pesados	9
1.10. Tolerancia a fallos y Observabilidad	9
1.11. Límites del sistema	10
1.11.1. Dentro del alcance	10
1.11.2. Fuera del alcance	10
1.11.3. Entradas externas	10
1.11.4. Salidas externas	11
2. Metas y Restricciones de la Arquitectura	11
2.1. Requisitos de los Stakeholders	11
2.2. Atributos de Calidad	12
2.3. Supuestos y limitaciones derivadas	13
2.3.1. Restricciones de arquitectura	13
2.3.2. Supuestos de operación (Evolución de v1.0)	13
3. Principios y decisiones arquitectónicas	14
3.1. Estilo arquitectónico: microkernel con núcleo activo	15
3.2. Principios arquitectónicos	16
3.2.1. Núcleo y contratos	16
3.2.2. Estado y memoria	17
3.2.3. Concurrencia	17
3.2.4. Tolerancia a fallos y observabilidad	18
3.3. Decisiones arquitectónicas	18
3.3.1. Modelo de concurrencia del TaskOrchestrator	20
3.4. Patrones aplicados	21

3.5. Trazabilidad entre el diagnóstico y las decisiones	21
4. Vistas Arquitectónicas (4+1)	22
4.1. Vista de Escenarios	22
4.1.1. Propósito	22
4.1.2. Notación	22
4.1.3. Visión general del modelo	23
4.1.4. Subsistema Interfaz y Usabilidad	24
4.1.5. Subsistema Núcleo y Ciclo de Vida	25
4.1.6. Subsistema Preprocesamiento	26
4.1.7. Subsistema Análisis	27
4.1.8. Subsistema Exportación	28
4.2. Vista Lógica	29
4.2.1. Propósito	29
4.2.2. Notación	29
4.2.3. Visión general del modelo	29
4.2.4. Elementos del núcleo	30
4.2.5. Servicios del sistema	30
4.2.6. Elementos de presentación	32
4.2.7. Contrato de extensión	32
4.2.8. Extensiones	33
4.2.9. Modelo de datos	33
4.2.10. Relaciones estructurales	34
4.2.11. Justificación de las decisiones	34
4.2.12. Trazabilidad con los requerimientos	35
4.3. Vista de Procesos	35
4.4. Vista Física	37
5. Atributos de calidad	38
5.1. Escenarios de calidad	38
5.1.1. Escenario de Calidad 1: Integración de un Plugin Externo	38
5.1.2. Escenario de Calidad 2: Carga de Señal de Gran Tamaño	39
5.1.3. Escenario de Calidad 3: Manejo de Errores en Tiempo de Ejecución	39
5.1.4. Escenario de Calidad 4: Exportación de Datos o Gráficas	40
5.1.5. Escenario de Calidad 5: Uso del Sistema sin Experiencia Técnica	40
5.1.6. Escenario de Calidad 6: Rendimiento con Múltiples Señales Cargadas	41
5.1.7. Escenario de Calidad 7: Resiliencia ante Archivos Corruptos o Incompatibles	41

5.1.8. Escenario de Calidad 8: Consistencia entre Datos Ingresados y Gráficas Generadas	42
5.2. Reglas de compatibilidad y versionado	42
5.3. Métricas y plan de verificación	43
5.4. Plan de evidencia	44
5.5. Conclusión	44
6. Anexos	45
6.1. Anexo A — Relación con el SRS	45
6.2. Anexo B — Relación con el SDD	45
6.3. Anexo C — Trazabilidad global de artefactos	45
6.4. Anexo D — Diagramas y tablas de referencia	45
7. Referencias	45

Índice de figuras

1.	Arquitectura microkernel de Gamma Lab 2.0: el núcleo y sus servicios, la capa de presentación y las extensiones conectadas mediante el contrato <code>IPlugin</code>	16
2.	Diagrama de casos de uso del sistema completo Gamma Lab 2.0 (23 CU en 5 subsistemas). Las relaciones <i>include/extend</i> se tomaron literalmente del campo “Extensiones” de cada ficha; ninguna es inferida por similitud temática.	23
3.	Diagrama de clases del sistema Gamma Lab 2.0. Los elementos en verde se encuentran implementados y los elementos en rojo están planificados.	30
4.	Flujo maestro de sesión de Gamma Lab 2.0, con carriles Investigador/Sistema. Verde = ya existe, rojo = nuevo, blanco = mixto.	36
5.	Vista física de Gamma Lab 2.0: estación única de escritorio y su superficie de E/S.	37

Índice de cuadros

1.	Acrónimos y términos utilizados en el documento	6
2.	Stakeholders principales y expectativas	11
3.	Síntesis del diagnóstico de Gamma Lab 1.0	14
4.	Decisiones arquitectónicas de Gamma Lab 2.0	20
5.	Patrones arquitectónicos aplicados en Gamma Lab 2.0.	21
6.	Trazabilidad entre el diagnóstico, los principios y las decisiones	22
7.	Casos de uso del subsistema Interfaz y Usabilidad	24
8.	Casos de uso del subsistema Núcleo y Ciclo de Vida	25
9.	Casos de uso del subsistema Preprocesamiento	26
10.	Casos de uso del subsistema Análisis	27
11.	Casos de uso del subsistema Exportación	28
12.	Extensiones representadas en la vista lógica.	33
13.	Relaciones estructurales de la vista lógica.	34
14.	Trazabilidad entre elementos de la vista lógica y requerimientos.	35
15.	Escenario de calidad 1 — Integración de un plugin externo	38
16.	Escenario de calidad 2 — Carga de señal de gran tamaño	39
17.	Escenario de calidad 3 — Manejo de errores en tiempo de ejecución	39
18.	Escenario de calidad 4 — Exportación de datos o gráficas	40
19.	Escenario de calidad 5 — Operación por un usuario sin experiencia técnica	40
20.	Escenario de calidad 6 — Rendimiento con múltiples señales cargadas	41
21.	Escenario de calidad 7 — Resiliencia ante archivos corruptos o incompatibles . .	41
22.	Escenario de calidad 8 — Consistencia entre datos ingresados y gráficas generadas	42
23.	Versiones de dependencias externas ancladas en <code>requirements.txt</code>	43
24.	Métricas, método de verificación y criterio de aceptación por atributo de calidad .	43

1. Introducción

1.1. Propósito

El presente documento tiene como propósito definir y documentar la arquitectura de GammaLab v2.0, estableciendo los principios, decisiones y lineamientos que orientan su evolución respecto a GammaLab v1.0. Para ello, describe la estructura arquitectónica de la solución, sus componentes principales, las relaciones entre ellos y las decisiones que permiten soportar la incorporación de nuevas capacidades de análisis de señales EEG, mejorar el rendimiento del sistema y facilitar su evolución y extensibilidad. Asimismo, presenta las vistas arquitectónicas bajo el modelo 4+1 y establece los criterios necesarios para evaluar que la arquitectura propuesta responda a los atributos de calidad definidos para el sistema, manteniendo coherencia con los requerimientos y objetivos establecidos para GammaLab v2.0.

1.2. Alcance

Este documento abarca la definición de los principios y decisiones arquitectónicas que rigen la estructura de Gamma Lab v2.0. La arquitectura adopta un estilo Microkernel, compuesto por un núcleo de servicios esenciales y un conjunto de plugins que extienden las capacidades del sistema mediante componentes desacoplados. Asimismo, se describen las vistas arquitectónicas bajo el modelo 4+1, los componentes y sus relaciones, los atributos de calidad, las estrategias para alcanzarlos y las métricas utilizadas para su evaluación. Finalmente, se documentan los principales riesgos y decisiones arquitectónicas y su trazabilidad con el SRS de Gamma Lab.

Fuera del alcance de este documento quedan los aspectos relacionados con la administración del proyecto, la implementación detallada de los componentes, los procedimientos de pruebas y las instrucciones de uso del sistema, los cuales se encuentran documentados en los artefactos correspondientes.

1.3. Definiciones y acrónimos

Cuadro 1: Acrónimos y términos utilizados en el documento

Sigla / Término	Definición
SAD	Software Architecture Document. Documento de arquitectura del sistema.
SRS	Software Requirements Specification. Documento de especificación de requisitos.
SDD	Software Design Document. Documento de diseño técnico.
IoC	Inversión de Control. Principio que delega el flujo de ejecución a un contenedor.
DI	Dependency Injection. Técnica para resolver dependencias desde un contenedor externo.
MVP / MVC	Patrones de separación de interfaz y lógica (Model–View–Presenter / Model–View–Controller).
Template Method	Patrón que define un flujo general con pasos sobrescribibles.
Adapter	Patrón para estandarizar interfaces incompatibles.
Caching	Estrategia para evitar recalcular o recomponer estructuras de datos.

Sigla / Término	Definición
Python	Lenguaje de programación principal de <i>Gamma Lab</i> .
PyQt5	Framework de interfaz gráfica basado en Qt para Python.
VTK	Visualization Toolkit. Biblioteca para visualización científica 2D/3D.
QVTK	Interfaz <code>QVTKRenderWindowInteractor</code> para incrustar VTK en aplicaciones Qt.
NumPy	Librería para cálculo numérico vectorizado.
EDF	European Data Format. Formato estándar para señales biomédicas.
ABF	Axon Binary File. Formato de archivo utilizado por Axon Instruments para registros biomédicos.
GUI Toolkit	Conjunto de herramientas para construir interfaces gráficas (Qt, PyQt, VTK).

1.4. Audiencia

Este documento está dirigido al equipo de desarrollo, así como a cualquier stakeholder con formación técnica que requiera comprender las decisiones arquitectónicas del sistema. Asimismo, está orientado a todos los investigadores y personas del Laboratorio de Electrofisiología de la Universidad de Colombia que estén interesados en conocer la estructura interna, los principios de diseño y los mecanismos de GammaLab v2.0 para el uso, mantenimiento y futuros cambios.

1.5. Referencias relacionadas

Este SAD se alinea con:

- **SRS de Gamma Lab:** requisitos funcionales y no funcionales del sistema.

Las referencias técnicas externas (PyQt, VTK, NumPy, EDF/ABF, YAML).

1.6. Visión General

Gamma Lab está construido sobre una arquitectura de microkernel, en la que un núcleo mínimo (Kernel) coordina la aplicación y la funcionalidad de dominio se delega en plugins independientes. El núcleo actúa como registro de servicios y de extensiones, y ofrece además un canal de eventos para comunicar los componentes. Los 16 plugins actuales se descubren automáticamente durante el arranque a partir de un descriptor declarativo (`properties.yml`). Cada uno implementa el contrato común `IPlugin`, y todos acceden al estado de la aplicación a través de un servicio compartido, el `DataStore`.

Teniendo en cuenta que Gamma Lab 2.0 parte de una base funcional y arquitectónica ya construida, se llevó a cabo un diagnóstico de la arquitectura inicial con el propósito de evaluar su estado actual, identificar las limitaciones de la implementación existente y reconocer oportunidades de mejora. Los resultados de este análisis se presentan en las siguientes secciones.

1.7. Núcleo con pocas responsabilidades y contratos débiles

En Gamma Lab 1.0 el núcleo se limita a ser un registro pasivo: mantiene dos diccionarios, uno de plugins y otro de servicios, y un canal de eventos genérico. No asume responsabilidades propias

del estilo microkernel, como gestionar el ciclo de vida de las extensiones, mediar su comunicación o verificar sus dependencias. El resultado es un *anemic kernel*: el núcleo existe estructuralmente, pero no gobierna el comportamiento del sistema en tiempo de ejecución.

La carencia más visible es la ausencia de gestión del ciclo de vida. Aunque `Kernel` dispone de `stop_plugins()`, ningún componente lo invoca y no existen estados ni transiciones que permitan controlar las extensiones. En consecuencia, el cierre de la aplicación no garantiza la liberación ordenada de recursos como contextos gráficos e hilos en ejecución, ni es posible descargar o recargar una extensión sin reiniciar el sistema.

La comunicación presenta una limitación similar. El núcleo ofrece un canal de eventos basado en señales de Qt, pero su firma genérica (`str`, `object`) permite tópicos arbitrarios y cargas útiles sin tipo definido. Además, solo transporta dos eventos, por lo que la coordinación restante se resuelve mediante referencias directas entre componentes. El mecanismo de eventos, que debería contribuir al desacoplamiento, termina teniendo un papel marginal.

En cuanto a los contratos, `IPlugin` concentra la lógica de procesamiento, la construcción de la interfaz gráfica y el acceso a los datos, violando el principio de responsabilidad única. Al quedar el cálculo ligado a objetos de presentación, no puede probarse ni ejecutarse de forma aislada, por ejemplo en un hilo de trabajo o proceso independiente.

Finalmente, los servicios se obtienen mediante un *service locator* basado en cadenas literales (`get_service("DataStore")`), por lo que las dependencias quedan ocultas en la implementación y no se verifican durante la carga. Un error tipográfico o un servicio ausente se manifiesta únicamente en tiempo de ejecución. De forma similar, registrar un plugin con un nombre existente sobrescribe silenciosamente al anterior, permitiendo que errores de configuración permanezcan ocultos hasta que se utiliza la funcionalidad afectada.

1.8. Estado y memoria

El estado de la aplicación reside en el `DataStore`, un diccionario global y mutable al que cualquier plugin puede acceder y modificar. Como ningún componente es responsable del ciclo de vida, cada archivo abierto permanece en memoria hasta el cierre de la aplicación, junto con sus datos derivados. En consecuencia, el consumo de memoria crece de forma monótona durante la sesión, y el usuario, tras abrir varios archivos o repetir segmentaciones, percibe un deterioro progresivo del rendimiento que solo se resuelve reiniciando el programa.

A esta retención se suma la replicación de los datos. Las señales se cargan completas en memoria, sin acceso parcial ni bajo demanda, y buena parte del flujo opera en doble precisión (`float64`). Sobre esa base se generan copias adicionales:

- La conversión a una tabla de VTK para la visualización realiza una copia profunda de la señal y la conserva asociada al conjunto de datos.
- La caché de ensayos filtrados crea una nueva matriz cada vez que se descartan ensayos, aunque el código asume lo contrario.
- La lectura de archivos EDF con frecuencias de muestreo heterogéneas remuestrea cada canal sobre un nuevo arreglo.

Esta mutabilidad sin control compromete la consistencia del estado. El proceso de remoción de artefactos modifica la matriz de ensayos en el mismo lugar (*in-place*), desde un hilo secundario y accediendo a un atributo privado de `SignalDataset`, lo que rompe su encapsulamiento. Como la caché de ensayos filtrados solo se invalida cuando cambia el conjunto de descartes, puede seguir

entregando una copia anterior a la corrección. En ese caso, los análisis posteriores operarían sobre datos que ya no corresponden a los que el usuario corrigió, sin que nada lo advierta. Además, como la modificación es destructiva y no conserva versiones previas, el usuario no puede deshacerla.

Por último, los recursos gráficos siguen la misma lógica de retención. Cada plugin crea su propia ventana de renderizado VTK sobre un contexto OpenGL compartido, y la ventana principal conserva todos los widgets instanciados hasta el cierre de la aplicación. Al desactivarse, un plugin solo deshabilita la interacción de su vista y no libera los buffers gráficos ni las tablas de datos asociadas, por lo que cada análisis que el usuario abre aumenta de forma permanente el consumo de memoria de video y de sistema.

1.9. Concurrencia y procesos pesados

GammaLab 1.0 no define un modelo de concurrencia a nivel arquitectónico. Por defecto, todo el procesamiento se ejecuta de forma síncrona en el hilo principal, el mismo que atiende el bucle de eventos de la interfaz gráfica. Por lo que, ante operaciones de varios segundos o minutos, el usuario no puede distinguir un procesamiento en curso de un fallo.

La concurrencia aparece solo como una decisión local de cada plugin. De los 16 plugins, únicamente dos delegan su cálculo en un hilo secundario, y cada uno lo hace con un patrón distinto:

- El promedio de *wavelets* hereda de `QThread`.
- La remoción de artefactos traslada un objeto de trabajo con `moveToThread`.

No existe una abstracción común para ejecutar tareas, ni un orquestador que las encole o limite cuántas se ejecutan simultáneamente. El sistema no impide que el usuario lance en paralelo varios procesos costosos desde distintos plugins. Cuando eso ocurre, compiten por los mismos núcleos de CPU y sus picos de memoria se suman, lo que agrava las condiciones de agotamiento descritas en la sección anterior.

La cancelación está igualmente limitada. Solo el promedio de *wavelets* la contempla, de forma cooperativa y entre un ensayo y el siguiente. Su rutina de limpieza invoca `quit()`, que no tiene efecto sobre un hilo sin bucle de eventos propio, y luego `wait()` desde el hilo principal, lo que congela la interfaz hasta que termine el ensayo en curso. Además, esa rutina está deshabilitada en el método de detención del plugin. Si el usuario cierra el plugin o la aplicación durante el cálculo, el hilo sigue ejecutándose sobre objetos que pueden estar siendo destruidos, y el cierre puede terminar en un error en lugar de una finalización ordenada.

Por último, el sistema carece de control de admisión. Antes de ejecutar un proceso no se estima su costo en tiempo ni en memoria, lo que puede generar que una configuración que excede los recursos del equipo no se rechace ni se advierta. Se ejecuta hasta congelar la aplicación o hasta fallar por falta de memoria, a veces tras varios minutos de cálculo cuyo resultado se pierde por completo.

1.10. Tolerancia a fallos y Observabilidad

Gamma Lab se ejecuta en un único proceso sin fronteras de aislamiento entre ellos. Un error nativo en VTK, un `MemoryError` o una excepción no capturada dentro de un *slot* de Qt terminan con la aplicación completa, porque no existe un manejador global (`sys.excepthook`) que lo impida. El usuario pierde así todas las señales, segmentaciones y correcciones de la sesión. El cargador omite en silencio los plugins que fallan al importarse, y el núcleo mantiene registrados aquellos cuya inicialización falló. Los errores no se corrigen, solo se ocultan: el sistema continúa en un estado

inconsistente y el usuario se encuentra con funcionalidades ausentes o con comportamientos erráticos, sin una explicación.

Estas fallas son difíciles de diagnosticar por la ausencia de observabilidad. El sistema no utiliza un mecanismo de registro estructurado. La información se emite con llamadas a `print`, sin niveles de severidad, marcas de tiempo ni identificación del componente de origen. Además, como la aplicación se ejecuta desde una consola, esa salida no se persiste y se pierde justamente cuando ocurre un cierre inesperado. Ante un fallo reportado no existe un registro que permita reconstruir qué operación se ejecutaba ni en qué estado estaba el sistema, por lo que el diagnóstico depende de reproducir manualmente el error.

1.11. Límites del sistema

1.11.1. Dentro del alcance

- **Ingreso y lectura:** Carga nativa de registros electroencefalográficos (EEG) almacenados localmente en formatos estándares .edf, .abf, .bdf y .mat.
- **Procesamiento y segmentación:** Aplicación de filtros digitales de corte, remoción de artefactos de estimulación y segmentación de señales en ensayos (trials) mediante umbrales en mV u operaciones de intervalo interestímulo (ISI).
- **Procesamiento espectral y tiempo-frecuencia:** Cálculo de promedios de Potenciales Relacionados a Eventos (ERP) con visualización de Butterfly plot bajo demanda, análisis de Transformada Rápida de Fourier (FFT), Densidad Espectral de Potencia (PSD) y Transformada Wavelet Continua con normalización y escalas personalizables.
- **Análisis de conectividad y acoplamiento avanzado:** Ejecución del análisis de acoplamiento fase-amplitud (PAC) tanto de un canal como cruzado intercanal.
- **Persistencia del espacio de trabajo:** Guardado y restauración del estado de la sesión analítica mediante archivos de proyecto local con extensión personalizada .glab.
- **Exportación de resultados:** Generación de archivos de datos numéricos estructurados (CSV con ejes X simplificados, Excel, JSON) y representaciones gráficas de alta resolución, incluyendo formato de imagen estándar (PNG, JPG, TIFF) e imagen vectorial científica (EMF).

1.11.2. Fuera del alcance

- La comunicación o adquisición de señales EEG en tiempo real mediante conexiones directas a equipos de amplificación, electrodos o diademas inalámbricas.
- La persistencia de proyectos en bases de datos centralizadas, servicios en la nube, servidores remotos o API web distribuidas; toda la persistencia es local e individual.
- El soporte operativo para plataformas web, sistemas móviles o sistemas operativos Unix/macOS en distribución comercial nativa (el instalador standalone se restringe a Windows de 64 bits).
- Mecanismos de control de acceso multiusuario, autenticación de seguridad en red, cifrado de datos clínicos o sincronización colaborativa síncrona.

1.11.3. Entradas externas

- Señales neurofisiológicas digitalizadas en formatos binarios compatibles (.edf, .abf, .bdf, .mat).
- Configuración alfanumérica de entrada por el usuario clínico (frecuencias de muestreo,

umbrales de trial en mV, bandas de frecuencia de fase/amplitud para PAC/PLV y ciclos de wavelet).

- Eventos de interacción directa del investigador con la GUI mediante teclado y ratón para realizar zoom multidireccional, desplazamiento temporal (pan) y selección de puntos específicos sobre las gráficas para la toma de medidas.

1.11.4. Salidas externas

- Gráficas 2D renderizadas mediante VTK embebido en PySide6, tales como comodulogramas de PAC, mapas de calor de coherencia temporal-frecuencial y espectrogramas Wavelet.
- Archivos con metadatos y estados serializados .glab que contienen los análisis intermedios, rutas absolutas y configuraciones del proyecto.
- Tablas de resultados analíticos en formato Excel (.xlsx), JSON y archivos CSV optimizados para evitar redundancia de ejes temporales.
- Archivos de imagen estándar (PNG/JPG/TIFF) y gráficos vectoriales de alta precisión (EMF) para uso directo en publicaciones indexadas.

2. Metas y Restricciones de la Arquitectura

2.1. Requisitos de los Stakeholders

Cuadro 2: Stakeholders principales y expectativas

Stakeholder	Tipo	Necesidades / Intereses
Laboratorio de Electrofisiología de la Universidad de Colombia	Usuario final	Su interés principal es contar con una herramienta robusta, intuitiva y libre de licencias que simplifique el análisis complejo por medio de PAC, sin necesitar habilidades de programación.
Equipo de desarrollo	Interno	Implementar una arquitectura extensible (microkernel + plugins), reestructurar el diseño de las pantallas aplicando estándares de codificación limpia y utilizar paralelización asíncrona.
Directora de tesis	Interno	Validar la rigurosidad científica de los algoritmos de frecuencia y supervisar el cumplimiento del alcance metodológico.
Evaluadores académicos	Externo / Validador	Verificar el cumplimiento de requerimientos, funciones y metodología aplicada desde la ingeniería de sistemas e ingeniería electrónica, demostrando cuantitativamente la mejora de rendimiento con respecto a GammaLab v1.0.
Comunidad científica	Externo / Colaborador	Posibilidad de reutilizar y mejorar el código abierto; transparencia metodológica y reproducibilidad de resultados; estandarización de flujos de trabajo y formatos de análisis EEG.

2.2. Atributos de Calidad

La arquitectura de **GammaLab 2.0** fue diseñada pensando principalmente en las necesidades manifestadas en el laboratorio de electrofisiología, entendiendo sus flujos de trabajo y los requerimientos propios de la ejecución de análisis experimentales. Al mismo tiempo, se proyecta como un software adaptable a distintos contextos y paradigmas de investigación, por lo que resulta fundamental considerar el impacto de los siguientes atributos de calidad en esta segunda versión:

1. **Extensibilidad:** Dado que el sistema debe adaptarse tanto a diferentes tipos de experimentos como a diversos entornos de investigación, la extensibilidad se concibe como una capacidad primordial. El diseño permite agregar nuevos módulos personalizados, completamente programables por cada laboratorio o usuario, y ofrece una interfaz clara para integrarlos de manera sencilla dentro del sistema. En **GammaLab 2.0**, esta capacidad se optimiza al desacoplar los plugins del núcleo mediante un contenedor de **Inversión de Control (IoC)**, registrando servicios comunes (como **ExportService**, **MeasurementService** y **SettingsService**) que evitan importaciones directas o acoplamientos rígidos. Asimismo, la entidad **SignalDataset** expone una API pública de comunicación estricta que encapsula la manipulación de trials, garantizando que futuras extensiones no alteren la integridad de los datos en memoria.
2. **Usabilidad:** GammaLab se privilegia de un diseño de interfaz de usuario pensado para ser lo más sencillo y entendible posible en cada funcionalidad implementada. Lo anterior resulta especialmente importante en el caso de los plugins, donde se manejan procesos conceptualmente complejos. En esta nueva versión, se eliminan las barreras técnicas de uso mediante ayudas contextuales integradas (*tooltips*) para todos los parámetros, menús unificados de edición en la vista principal (*Home*) y la representación explícita de gráficas complejas como el *Butterfly plot*. Adicionalmente, el sistema asegura la retroalimentación constante del estado del sistema mediante barras de progreso no bloqueantes para operaciones superiores a 1 segundo, garantizando un flujo intuitivo y controlado.
3. **Mantenibilidad:** La arquitectura propuesta favorece la modularidad y el ordenamiento claro del código, buscando que el sistema siga principios como el de Abierto/Cerrado dentro del marco SOLID: abierto a extensiones y cerrado a modificaciones innecesarias. El sistema realiza una separación rigurosa entre la interfaz gráfica (PySide6) y la lógica matemática en hilos de cómputo independientes, facilitando la depuración del código. Además, la mantenibilidad y la robustez técnica se garantizan mediante el apego estricto a las directrices de estilo **PEP8** y un estándar mínimo del **80 % de cobertura de pruebas unitarias automatizadas** utilizando el framework **Pytest**.
4. **Rendimiento:** El contexto de uso del programa exige un rendimiento estable y un consumo controlado de recursos, especialmente dada la naturaleza de los datos involucrados, que suelen tener alta densidad de información. Para dar soporte estable a señales de hasta 600 MB en memoria, **GammaLab 2.0** incorpora una estrategia de paralelización en el Kernel. Las tareas pesadas de procesamiento (PAC, Wavelet, PSD) se distribuyen de forma asíncrona mediante multiprocesamiento (**multiprocessing**), impidiendo que la interfaz gráfica se bloquee durante los análisis (manteniendo la responsividad ≤ 0.3 s). Adicionalmente, se introduce el submuestreo dinámico visual a un límite de 2000 puntos por serie en los widgets de VTK para conservar una navegación fluida (≥ 24 FPS) sin degradar la data cruda, acompañado de rutinas explícitas de liberación de memoria de GPU y RAM en cada plugin al cambiar de módulo.
5. **Portabilidad:** El sistema debe asegurar su correcta ejecución en diversos entornos locales sin requerir la intervención técnica continua del equipo desarrollador. **GammaLab 2.0**

centraliza su portabilidad local en sistemas Windows 10/11 de 64 bits. Este atributo se consolida al empaquetar la aplicación de manera autónoma en un **ejecutable standalone (.exe)** a través de PyInstaller. Esto permite que el personal clínico e investigadores ejecuten análisis avanzados localmente con un doble clic, omitiendo por completo la configuración o instalación manual de librerías científicas y consolas de Python en sus estaciones de trabajo.

6. **Robustez y Fiabilidad:** El sistema debe mantener la estabilidad operativa en sesiones continuas y tolerar errores imprevistos en tiempo de ejecución. Ante excepciones internas generadas por plugins de terceros o formatos de archivo corruptos, el sistema captura los fallos de manera centralizada sin interrumpir la sesión del usuario ni provocar la caída de la aplicación. Adicionalmente, el sistema implementa exclusiones mutuas para accesos concurrentes al almacén de datos centralizado (*DataStore*), asegura la correcta liberación de observadores y callbacks VTK y proporciona terminaciones controladas (*cleanup*) de hilos de procesamiento de datos ante solicitudes explícitas de cancelación.
7. **Trazabilidad y Persistencia:** El sistema incorpora en cada análisis la información contextual y los metadatos necesarios para asegurar la reproducibilidad y trazabilidad de los resultados. **GammaLab 2.0** materializa esta capacidad de forma completa mediante el módulo **PJ (Project Session)**, el cual introduce la persistencia local del estado completo del espacio de trabajo (rutas de señales, trials configurados y análisis procesados) en archivos con extensión de proyecto **.glab** o **.glws**. Asimismo, la trazabilidad científica externa se respalda al exportar resultados consistentes en formatos tabulares (CSV sin redundancia de ejes temporales, Excel, JSON) y gráficos de alta calidad vectorial en formato **EMF**.

1.

2.3. Supuestos y limitaciones derivadas

A continuación se presentan los supuestos y limitaciones considerados durante el diseño y construcción del sistema:

2.3.1. Restricciones de arquitectura

- **Operación standalone y offline:** El sistema debe ejecutarse de forma local en la máquina del investigador de forma estricta. No se permite el uso de servicios en la nube, APIs de procesamiento externas ni el uso de bases de datos distribuidas.
- **Entorno tecnológico base:** El núcleo matemático y la lógica de interacción deben desarrollarse bajo el lenguaje Python (versión mínima 3.11), utilizando Pyside6 para la interfaz y VTK para la visualización científica.
- **Compatibilidad de datos:** El módulo de entrada/salida soporta y procesa archivos de señales en formatos como: EDF, ABF, BDF, MAT y GLAB.

2.3.2. Supuestos de operación (Evolución de v1.0)

- **Procesamiento concurrente:** En esta nueva versión 2.0 se asume que el hardware cuenta con arquitectura multinúcleo (más de 4 núcleos) para el despliegue de procesamiento asíncrono y multiprocesamiento en segundo plano.
- **Naturaleza del proyecto:** El sistema está diseñado exclusivamente para el posprocesamiento y análisis de señales previamente adquiridas. No soporta adquisición de datos en tiempo real ni comunicación con hardware de amplificación o registro fisiológico.

3. Principios y decisiones arquitectónicas

La arquitectura de Gamma Lab 2.0 no parte de cero. Conserva el estilo microkernel de la versión 1.0, pero corrige las limitaciones identificadas en el diagnóstico de las secciones anteriores. Por esta razón, los principios y las decisiones de esta sección se formulan como respuesta a problemas concretos y no como aspiraciones generales. La siguiente tabla sintetiza esos problemas y les asigna un identificador, de D1 a D15, que se utiliza en el resto de la sección para trazar cada principio y cada decisión hasta su origen.

ID	Problema
D1	El núcleo es un registro pasivo: no gestiona el ciclo de vida de las extensiones, <code>stop_plugins()</code> nunca se invoca y no es posible descargar ni recargar una extensión.
D2	El canal de eventos tiene una firma genérica (<code>str</code> , <code>object</code>), transporta solo dos eventos y la coordinación restante se resuelve con referencias directas entre componentes.
D3	<code>IPlugin</code> concentra procesamiento, interfaz gráfica y acceso a datos, por lo que el cálculo no puede probarse ni ejecutarse de forma aislada.
D4	Los servicios se obtienen con un <i>service locator</i> basado en cadenas literales: las dependencias quedan ocultas, no se verifican en la carga y un registro duplicado sobrescribe al anterior.
D5	El <code>DataStore</code> es un diccionario global y mutable sin un responsable de su ciclo de vida, por lo que la memoria crece de forma monótona durante la sesión.
D6	Los datos se replican: carga completa en <code>float64</code> , copia profunda hacia VTK, nuevas matrices en la caché de ensayos y remuestreo de canales EDF.
D7	La remoción de artefactos modifica los ensayos <i>in-place</i> desde un hilo secundario, rompe el encapsulamiento de <code>SignalDataset</code> , deja cachés obsoletas y no permite deshacer.
D8	Cada plugin conserva su ventana de renderizado VTK y sus tablas; al desactivarse no libera los recursos gráficos.
D9	No existe un modelo de concurrencia: el procesamiento se ejecuta en el hilo de la interfaz y el usuario no distingue un cálculo en curso de un fallo.
D10	La concurrencia es una decisión local de cada plugin, con patrones distintos y sin un orquestador que limite las tareas simultáneas.
D11	La cancelación es parcial y defectuosa: bloquea la interfaz y el hilo puede seguir ejecutándose sobre objetos destruidos.
D12	No hay control de admisión: no se estima el costo en tiempo ni en memoria antes de ejecutar un proceso.
D13	Todo se ejecuta en un único proceso sin manejador global de excepciones: un error no capturado termina la aplicación y se pierde la sesión.
D14	Los errores se ocultan: el cargador omite los plugins que fallan al importarse y el núcleo mantiene registrados los que fallaron al inicializarse.
D15	No hay registro estructurado: la información se emite con <code>print</code> , sin niveles, sin marcas de tiempo y sin persistencia.

Cuadro 3: Síntesis del diagnóstico de Gamma Lab 1.0

3.1. Estilo arquitectónico: microkernel con núcleo activo

Gamma Lab 2.0 mantiene el estilo microkernel: un núcleo estable ofrece los servicios transversales y un conjunto de extensiones independientes implementa la funcionalidad de dominio, como el filtrado, la segmentación en ensayos, la FFT, la PSD, la transformada wavelet, el ERP y el PAC. El diagnóstico muestra que las limitaciones de la versión 1.0 no provienen del estilo, sino de su implementación: el núcleo existe estructuralmente, pero no gobierna el comportamiento del sistema (D1). Por ello, la decisión central de esta versión no es cambiar de estilo, sino asignar al núcleo las responsabilidades que el estilo le atribuye:

- Gobernar el ciclo de vida de las extensiones, desde su descubrimiento hasta su detención.
- Mediar la comunicación entre extensiones mediante eventos declarados.
- Centralizar la ejecución de los cálculos pesados fuera del hilo de la interfaz.
- Validar las extensiones y sus dependencias durante la carga.

Se consideraron dos alternativas. Reescribir el sistema como un monolito en capas habría simplificado el control del flujo, pero obligaba a reescribir las dieciséis extensiones existentes y sacrificaba la extensibilidad, que es una necesidad explícita del laboratorio. Un estilo de tuberías y filtros se ajusta bien al procesamiento de señales, pero no a una aplicación interactiva en la que el usuario navega entre módulos que comparten el mismo estado. El microkernel permite conservar las extensiones existentes y, al mismo tiempo, concentrar en el núcleo el control que hoy está disperso.

El núcleo no asume todas estas responsabilidades por sí mismo, sino que las distribuye entre un conjunto de servicios que no son reemplazables por extensiones:

- **Kernel**: mantiene el registro de extensiones y servicios, gobierna el ciclo de vida de las extensiones y ofrece el canal de eventos.
- **PluginManager**: descubre las extensiones a partir de su descriptor `properties.yml` y valida sus metadatos, la versión del contrato y sus dependencias antes de registrarlas.
- **TaskOrchestrator** (implementado como **TaskService**): encola y ejecuta los cálculos pesados fuera del hilo de la interfaz, informa el progreso, gestiona la cancelación y es el único componente que escribe en el estado compartido.
- **DataStore**: mantiene el estado compartido de la sesión, es decir, las señales, la señal activa, sus derivados y los resultados.
- **FileIOService**: adapta los formatos de archivo soportados al modelo de datos interno (`SignalDataset`).
- **ExportService**: exporta visualizaciones y resultados tabulares a formatos externos.
- **SettingsService**: gestiona la configuración persistente de la aplicación.
- **ProjectService**: persiste el espacio de trabajo en archivos `.glab` y es responsable del ciclo de vida del proyecto activo.

Alrededor del núcleo se organizan otras tres agrupaciones, coherentes con la vista lógica (Sección 4.2): la capa de presentación, construida con PyQt5 y VTK, que contiene la ventana principal y los elementos de interacción con las visualizaciones, como el menú contextual y el servicio de

mediciones asociado a cada vista; las extensiones, que realizan el contrato `IPlugin`; y el modelo de datos, formado por `SignalDataset`, `TrialDataset` y la estructura del proyecto. La Figura 1 presenta esta organización.

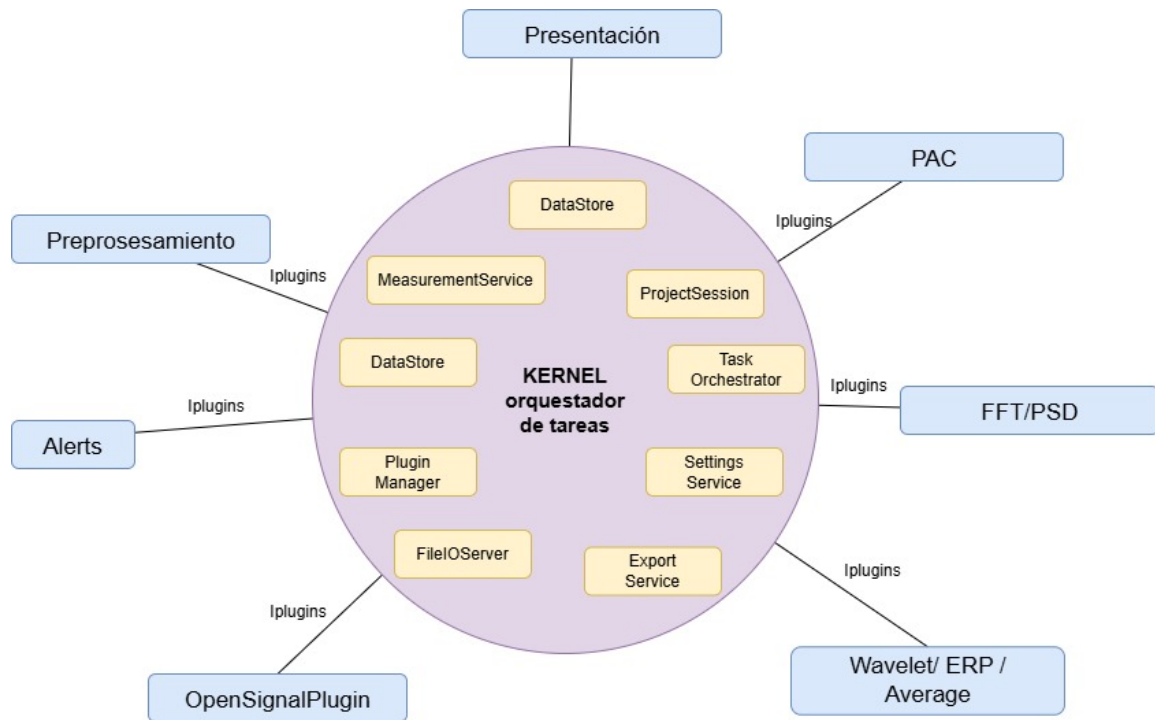


Figura 1: Arquitectura microkernel de Gamma Lab 2.0: el núcleo y sus servicios, la capa de presentación y las extensiones conectadas mediante el contrato `IPlugin`.

3.2. Principios arquitectónicos

Los principios se agrupan en las mismas cuatro áreas del diagnóstico. Cada uno se enuncia como una regla que el diseño debe cumplir e indica los problemas de la Tabla 4 a los que responde.

3.2.1. Núcleo y contratos

- **El núcleo gobierna el ciclo de vida de las extensiones:** Cada extensión pasa por estados explícitos: descubierta, inicializada, activa, inactiva, detenida o fallida. Solo el núcleo ejecuta las transiciones entre ellos. El cierre de la aplicación (CU-022) recorre las extensiones e invoca su detención, y una extensión cuya inicialización falla queda en estado fallida y visible para el usuario, en lugar de permanecer registrada como si funcionara. *Responde a: D1, D14.*
- **Las extensiones se comunican mediante eventos declarados:** Las extensiones no se referencian entre sí. Se coordinan a través de un catálogo cerrado de eventos del núcleo, por ejemplo, señal cargada, ensayos generados, conjunto derivado creado o tarea finalizada, y cada evento tiene una carga útil de estructura definida. Un evento que no pertenece al catálogo se rechaza al emitirse. Así, el canal de eventos deja de tener un papel marginal y se convierte en el mecanismo de desacoplamiento que el estilo prevé. *Responde a: D2.*
- **El cómputo se separa de la presentación:** El cálculo de cada extensión se implementa como una función sin dependencias de Qt ni de VTK, que recibe arreglos y parámetros y retorna un resultado. La extensión conserva la construcción de su vista y la solicitud de

ejecución al orquestador. Esta separación permite probar el cálculo de forma aislada contra las referencias de MATLAB y ejecutarlo en un hilo o en un proceso trabajador sin arrastrar objetos de presentación. *Responde a: D3.*

- **Las dependencias se declaran y se verifican durante la carga:** Cada extensión declara en su `properties.yml` la versión del contrato `IPlugin` con la que fue escrita y los servicios que requiere. Durante el arranque, el `PluginManager` verifica que la versión sea compatible, que los servicios existan y que el identificador no esté repetido. Si alguna condición no se cumple, la extensión no se carga y el usuario recibe la causa. Las claves de los servicios se definen en un único lugar, en vez de repetirse como cadenas literales. El *service locator* se conserva, pero sus errores pasan del tiempo de ejecución al arranque. *Responde a: D4, D14.*

3.2.2. Estado y memoria

- **El estado tiene un dueño y los datos originales no se modifican:** El `DataStore` tiene un único escritor, el `TaskOrchestrator` (R54): las extensiones lo consultan, pero no lo modifican directamente. Las transformaciones de preprocesamiento no alteran la señal ni los ensayos de origen, sino que producen conjuntos derivados que referencian a su fuente. Para que esta regla no multiplique el consumo de memoria, un derivado materializa solo lo que cambia; por ejemplo, el descarte de ensayos se representa con una máscara de índices y no con una nueva matriz. Cada conjunto tiene una versión, y las cachés que dependen de él se invalidan cuando esta cambia. Por último, el `ProjectService` es el responsable del ciclo de vida del proyecto, de modo que al cerrarlo o reemplazarlo (CU-021, CU-003) se liberan la señal y todos sus derivados. *Responde a: D5, D7.*
- **Ningún componente copia la señal sin necesidad:** La visualización trabaja sobre una representación submuestreada de la señal (CU-019) y no sobre una copia profunda completa. La selección de ensayos se resuelve con índices o vistas de NumPy. El tipo numérico se elige por etapa: se usa precisión simple donde la diferencia frente a las referencias de MATLAB está dentro de la tolerancia aceptada y precisión doble donde no lo está. *Responde a: D6.*
- **Los recursos se liberan de forma determinista:** Al desactivarse una extensión, por ejemplo al cambiar de módulo (CU-008), esta libera su ventana de renderizado, sus observadores y sus tablas de VTK, y su vista se reconstruye bajo demanda cuando el usuario regresa. Al cerrar el proyecto o la aplicación (CU-021, CU-022) no quedan hilos, procesos ni contextos gráficos activos. *Responde a: D5, D8.*

3.2.3. Concurrencia

- **La ejecución de los cálculos pesados está centralizada y es cancelable:** Ninguna extensión crea hilos ni procesos. Toda operación de cómputo pesado se solicita al `TaskOrchestrator`, que la encola, la ejecuta fuera del hilo de la interfaz y de a una a la vez (R70), presenta un indicador de progreso cuando supera un segundo y permite cancelarla (CU-014). La cancelación es cooperativa: el cómputo consulta periódicamente una señal de cancelación. Si no responde dentro de un tiempo acotado, el orquestador fuerza su terminación. *Responde a: D9, D10, D11.*
- **Antes de ejecutar se estima el costo:** El orquestador estima la memoria que requiere un cómputo a partir de las dimensiones de la entrada y de sus parámetros, por ejemplo, el número de canales, muestras y frecuencias de una transformada wavelet o de un análisis

PAC, y la compara con la memoria disponible. Si la estimación supera lo disponible, el sistema advierte al usuario y le propone reducir los parámetros; si la ejecución es claramente inviable, la rechaza antes de iniciarla. *Responde a: D12.*

3.2.4. Tolerancia a fallos y observabilidad

- **Los fallos se contienen y se hacen visibles:** Un fallo no debe terminar la aplicación ni pasar inadvertido. Un manejador global captura las excepciones no controladas del hilo principal y de los *slots* de Qt. Las excepciones de un cómputo se capturan en el trabajador y llegan a la interfaz como un resultado fallido. Los cómputos con riesgo de agotar la memoria o de fallar en código nativo se ejecutan en un proceso aislado, de modo que su caída no arrastra a la aplicación. En todos los casos el usuario recibe un mensaje comprensible y la sesión se conserva. *Responde a: D13, D14.*
- **El sistema es observable:** Todos los componentes registran sus eventos con nivel de severidad, marca de tiempo, componente de origen e identificador de la tarea cuando aplica. El registro se persiste en un archivo rotativo en el directorio del usuario, por lo que sobrevive a un cierre inesperado y permite reconstruir qué operación se ejecutaba cuando ocurrió un fallo. *Responde a: D15.*

3.3. Decisiones arquitectónicas

La Tabla ?? resume las decisiones arquitectónicas que materializan los principios anteriores. Para cada una se indica el problema del diagnóstico al que responde, la alternativa que se descartó, sus consecuencias y su estado: *implementada*, si ya está presente en el código; *parcial*, si existe una parte; o *planificada*, si debe construirse en esta versión.

ID	Decisión	D	Alternativa descartada	Consecuencias
ADR-01	Mantener el estilo microkernel y asignar al núcleo responsabilidades propias.	D1	Monolito en capas; tuberías y filtros.	Se conservan las extensiones y el contrato existentes. El núcleo pasa a ser un componente crítico que requiere pruebas propias.
ADR-02	Ciclo de vida de las extensiones como máquina de estados gobernada por el Kernel.	D1, D14	Que cada extensión gestione su propia activación, como en 1.0.	Cierre ordenado y extensiones fallidas visibles. Exige que toda extensión implemente una detención completa.
ADR-03	Catálogo de eventos con carga útil tipada.	D2	Canal genérico (<i>str</i> , <i>object</i>) y referencias directas.	Los errores de tópico se detectan al emitir. El catálogo se convierte en un contrato que debe versionarse.

Continúa en la siguiente página

ID	Decisión	D	Alternativa descartada	Consecuencias
ADR-04	Separar el cómputo de cada extensión en un módulo compute sin dependencias de Qt ni VTK.	D3	IPlugin único con lógica, vista y datos.	El cálculo es comprobable y ejecutable en un trabajador. Cada extensión pasa a tener dos módulos.
ADR-05	Descubrimiento declarativo validado: properties.yml con versión del contrato, dependencias y unicidad.	D4, D14	Contenedor de inyección de dependencias, por el costo de migrar todas las extensiones.	Los errores de configuración se detectan en el arranque. Se amplía el esquema del descriptor.
ADR-06	DataStore con escritor único y conjuntos derivados inmutables.	D5, D7	Escritura libre protegida con exclusión mutua; modificación <i>in-place</i> .	Estado consistente, trazable y con posibilidad de deshacer. Los derivados deben materializar solo lo que cambia.
ADR-07	Estrategia de memoria: representación submuestreada, sin copia profunda hacia VTK y tipo numérico por etapa.	D6	Carga perezosa o memoria mapeada, que depende del formato de cada archivo.	Menor consumo de memoria. La precisión debe validarse contra las referencias de MATLAB.
ADR-08	Liberación determinista de recursos al desactivar una extensión, cambiar de módulo y cerrar.	D5, D8	Conservar en caché todas las vistas instanciadas.	Consumo acotado de memoria de video y de sistema. Reconstruir una vista al regresar tiene un costo de tiempo.
ADR-09	TaskOrchestrator híbrido: hilos por defecto y proceso trabajador para los cómputos aislables (Sección 3.3.1).	D9–D11, D13	Solo hilos; solo multiprocesamiento.	Interfaz responsiva, cancelación forzada real y aislamiento de fallos. Costo de serializar arreglos grandes.
ADR-10	Control de admisión mediante la estimación del costo de cada cómputo.	D12	Ejecutar sin estimar, como en 1.0.	Se evitan congelamientos y cálculos perdidos. Cada extensión debe proveer su estimación.
ADR-11	Manejador global de excepciones y captura en los trabajadores con notificación a la interfaz.	D13, D14	Manejo local en cada extensión.	La sesión se conserva ante fallos. Los mensajes deben ser comprensibles para el usuario.

Continúa en la siguiente página

ID	Decisión	D	Alternativa descartada	Consecuencias
ADR-12	Registro estructurado con el módulo <code>logging</code> y archivo rotativo.	D15	Mensajes con <code>print</code> en consola.	Diagnóstico posterior a un fallo sin reproducirlo. Deben reemplazarse las llamadas existentes a <code>print</code> .
ADR-13	Python, PyQt5 y VTK como base tecnológica.	—	Migrar a PySide6.	Base madura para cálculo y visualización científica. La licencia GPL de PyQt5 condiciona la distribución, y VTK solo puede renderizar en el hilo principal.

Cuadro 4: Decisiones arquitectónicas de Gamma Lab 2.0

3.3.1. Modelo de concurrencia del TaskOrchestrator

La decisión ADR-09 merece una explicación adicional, porque es la que más condiciona la respuesta del sistema. El `TaskService` actual ya resuelve buena parte de D9 y D10: mantiene una cola de tareas, ejecuta una sola a la vez en un hilo trabajador, descarta las tareas pendientes de una extensión cuando esta solicita una nueva y ofrece a cada cómputo una señal de cancelación cooperativa. Cuando el usuario cancela una tarea que no responde en dos segundos, el servicio deja de esperarla y descarta su resultado, pero no puede detenerla. Esta limitación es propia de los hilos de Python, que no pueden terminarse de forma segura desde fuera: el hilo desacoplado sigue consumiendo CPU y memoria hasta terminar, y un `MemoryError` o un error nativo dentro de él sigue afectando a toda la aplicación.

Por ello, la terminación forzada que exige CU-014 y el aislamiento de fallos de P10 solo pueden lograrse con un proceso trabajador, que sí puede terminarse sin comprometer el proceso principal. Se adopta un modelo híbrido:

- **Hilo trabajador** para los cálculos breves o de bajo consumo de memoria, como el filtrado, la segmentación o el promedio de ensayos, donde el costo de crear un proceso y de transferir los datos superaría el beneficio.
- **Proceso trabajador** para los cálculos intensivos y aislables, como el PAC, la transformada wavelet y el índice de modulación, cuya duración y consumo de memoria justifican poder terminarlos y aislar su posible falla.

El modelo tiene dos costos que el diseño debe atender. El primero es la transferencia de arreglos grandes entre procesos, que se mitiga con memoria compartida en lugar de serialización completa y con un proceso trabajador persistente que evita el costo de arranque en cada tarea. El segundo es que el resultado siempre debe volver al hilo principal, porque VTK solo puede renderizar en él. En ambos casos, la extensión no conoce el mecanismo utilizado: entrega una función de cómputo (P3) y el orquestador decide cómo ejecutarla.

3.4. Patrones aplicados

La Tabla anterior resume los patrones que materializan los principios y las decisiones anteriores.

Patrón	Problema que resuelve	Aplicación en Gamma Lab 2.0
Microkernel / Plugin	Agregar capacidades sin modificar el núcleo.	Cada extensión declara un <code>properties.yml</code> y una clase que realiza <code>IPlugin</code> ; el <code>PluginManager</code> la descubre y la registra en el Kernel.
State	Gobernar el ciclo de vida de las extensiones (D1).	El Kernel mantiene el estado de cada extensión y solo permite las transiciones definidas (P1, ADR-02).
Service Locator	Ofrecer servicios compartidos sin que las extensiones los instancien (D4).	El Kernel expone los servicios por clave. Sus dependencias se declaran y se verifican en la carga para mitigar sus riesgos (P4, ADR-05).
Observer	Comunicar cambios sin referencias directas (D2).	El canal de eventos del Kernel, basado en señales de Qt, notifica a los suscriptores los eventos del catálogo (P2, ADR-03).
Command con señal de cancelación	Ejecutar, seguir y cancelar cálculos de forma uniforme (D10, D11).	Cada cómputo se encapsula como una tarea con su <code>TaskHandle</code> (progreso, finalización, fallo, cancelación) y su <code>TaskContext</code> (señal de cancelación cooperativa).
Single Writer	Mantener la consistencia del estado compartido (D5, D7).	Solo el <code>TaskOrchestrator</code> escribe en el <code>DataStore</code> (P5, ADR-06).
Adapter	Integrar representaciones incompatibles.	El <code>FileIOService</code> adapta los formatos ABF, EDF y MAT a <code>SignalDataset</code> ; los adaptadores de VTK convierten arreglos de NumPy en estructuras de visualización.
Strategy	Intercambiar extensiones detrás de un mismo contrato.	El núcleo y la ventana principal operan sobre <code>IPlugin</code> sin conocer la extensión concreta.
Template Method	Reutilizar el procedimiento común de los análisis de acoplamiento.	<code>PACAnalysisPlugin</code> define el cálculo del mapa de fase y frecuencia, y sus especializaciones de uno y dos canales completan los pasos propios.

Cuadro 5: Patrones arquitectónicos aplicados en Gamma Lab 2.0.

3.5. Trazabilidad entre el diagnóstico y las decisiones

La Tabla 6 cierra la sección relacionando cada problema del diagnóstico con el principio y la decisión que lo atienden, y con los casos de uso de la vista de escenarios y los escenarios de calidad en los que se verifica su solución.

D	Principio	Decisión	Verificación (caso de uso / escenario)
D1	P1	ADR-01, ADR-02	CU-011, CU-022
D2	P2	ADR-03	CU-003, CU-004
D3	P3	ADR-04	Escenario 1; métrica de mantenibilidad (cobertura)
D4	P4	ADR-05	CU-011; Escenario 1
D5	P5, P7	ADR-06, ADR-08	CU-003, CU-021; Escenario 6
D6	P6	ADR-07	CU-019; Escenarios 2 y 6
D7	P5	ADR-06	CU-007, CU-016; Escenario 8

D	Principio	Decisión	Verificación (caso de uso / escenario)
D8	P7	ADR-08	CU-008, CU-022
D9	P8	ADR-09	CU-020; Escenario 2
D10	P8	ADR-09	CU-020
D11	P8	ADR-09	CU-014; métrica de robustez ante cancelación
D12	P9	ADR-10	CU-020; Escenario 2
D13	P10	ADR-09, ADR-11	Escenarios 3 y 7
D14	P1, P4, P10	ADR-02, ADR-05, ADR-11	CU-011; Escenarios 1 y 3
D15	P11	ADR-12	Escenario 3

Cuadro 6: Trazabilidad entre el diagnóstico, los principios y las decisiones

4. Vistas Arquitectónicas (4+1)

4.1. Vista de Escenarios

4.1.1. Propósito

El diagrama de casos de uso describe el comportamiento observable de Gamma Lab 2.0 desde la perspectiva de su usuario. Representa las funcionalidades que el sistema ofrece, el actor que las utiliza y las relaciones de dependencia, extensión y especialización entre ellas. Cada caso de uso agrupa uno o más requerimientos funcionales, por lo que el modelo vincula la especificación de requerimientos con la estructura de la arquitectura. Además, al organizar los casos de uso en subsistemas, el diagrama anticipa la descomposición funcional del sistema y muestra cómo se distribuyen las responsabilidades entre sus componentes.

4.1.2. Notación

El diagrama sigue la notación de casos de uso definida en la especificación UML 2.5.1 del Object Management Group [17]. El sistema se representa como un contenedor general con el estereotipo «system». Dentro de él, los casos de uso se agrupan en paquetes con el estereotipo «subsistema», cada uno asociado a un área de responsabilidad funcional. Las relaciones entre casos de uso son de tres tipos:

- **Inclusión («include»)**: línea punteada que va del caso base al caso incluido. Indica que el comportamiento incluido es obligatorio y forma parte de la ejecución del caso base.
- **Extensión («extend»)**: línea punteada que va del caso extensor al caso base. Indica un comportamiento opcional o condicional que amplía el flujo del caso base sin modificarlo.
- **Generalización**: línea continua que va del caso especializado al caso general. Indica que el caso hijo hereda y especializa el comportamiento del padre.

Los casos de uso marcados con el estereotipo «abstract» no se ejecutan directamente. Su función es definir un comportamiento común que concretan sus especializaciones.

4.1.4. Subsistema Interfaz y Usabilidad

El subsistema **Interfaz y Usabilidad** agrupa los casos de uso que definen cómo interactúa el Investigador con Gamma Lab 2.0 a través de la capa de presentación. Su responsabilidad no es procesar señales, sino garantizar que la interacción sea comprensible, fluida y estable en el uso de recursos. Por eso concentra principalmente requerimientos no funcionales de usabilidad, rendimiento y confiabilidad, junto con algunos requerimientos funcionales de visualización. El Cuadro 7 resume los casos de uso que lo componen.

Cuadro 7: Casos de uso del subsistema Interfaz y Usabilidad

Caso de uso	Nombre	Descripción
CU-008	Navegar y cambiar de módulo	Permite desplazarse entre los módulos funcionales del sistema, liberando los recursos del módulo saliente y conservando la señal activa en memoria.
CU-009	Limpiar formulario (Clear)	Restablece a sus valores por defecto los parámetros de una funcionalidad, sin modificar los resultados ya presentados.
CU-010	Consultar ayuda contextual	Muestra una descripción emergente del propósito de cada control de la interfaz.
CU-012	Interactuar visualmente con la señal	Permite explorar señales y resultados mediante zoom, lectura de coordenadas y vista Butterfly, sin alterar los datos.
CU-019	Optimizar renderizado (submuestreo)	Reduce la densidad de la representación gráfica para mantener la fluidez del renderizado, sin modificar los datos en memoria.

El subsistema se organiza alrededor de CU-008, que funciona como eje de la interacción y como único punto de conexión con el subsistema Núcleo y Ciclo de Vida. Esta posición se debe a que el cambio de módulo es una transición en el ciclo de vida de las extensiones. Al desactivarse, cada módulo debe liberar sus recursos gráficos y sus detectores de eventos de forma determinista. Al activarse, el módulo destino debe reutilizar los componentes ya construidos, sin recargar la señal desde disco. Los demás casos de uso del subsistema se vinculan con este eje. CU-010 lo extiende como un comportamiento opcional, activado por el usuario, que no altera el estado del sistema. CU-019 se relaciona con la navegación porque la optimización del renderizado condiciona la fluidez de cualquier vista presentada.

El diagrama también muestra un segundo agrupamiento, formado por CU-012 y CU-019. CU-012 incluye a CU-019, lo que expresa que toda interacción visual sobre una señal pasa obligatoriamente por la estrategia de submuestreo de representación. Esta relación materializa la separación entre el modelo de datos y la vista: el zoom, el desplazamiento y la lectura de coordenadas modifican solo la representación, mientras que los datos en memoria permanecen intactos.

Por último, CU-009 aparece en el diagrama vinculado con CU-019 mediante una relación de extensión. Este caso de uso representa una operación sobre el estado del formulario de cada funcionalidad: restablece los parámetros a sus valores por defecto sin modificar los resultados ya presentados ni los datos en memoria. Su ejecución está condicionada a que no exista un cómputo en curso lanzado desde ese formulario, lo que mantiene la coherencia entre los parámetros mostrados y la tarea en ejecución.

En conjunto, el subsistema resuelve dentro de la capa de presentación los atributos de calidad más sensibles para el usuario: la capacidad de respuesta, la fluidez del renderizado y el consumo

acotado de memoria y recursos gráficos. Al concentrar su conexión con el resto del sistema en CU-008, mantiene la lógica de interacción desacoplada de la lógica de procesamiento y delega la ejecución de las operaciones pesadas en el orquestador del núcleo.

4.1.5. Subsistema Núcleo y Ciclo de Vida

El subsistema **Núcleo y Ciclo de Vida** materializa el estilo arquitectónico de microkernel. En él se registran los servicios compartidos, se descubren las extensiones y se centraliza la ejecución de las operaciones pesadas, por lo que concentra requerimientos de rendimiento, confiabilidad y soporte. El Cuadro 8 resume los casos de uso que lo componen.

Cuadro 8: Casos de uso del subsistema Núcleo y Ciclo de Vida

Caso de uso	Nombre	Descripción
CU-011	Iniciar la aplicación	Arranca el sistema, registra en el núcleo los servicios compartidos y descubre las extensiones disponibles.
CU-022	Cerrar la aplicación	Finaliza la ejecución liberando hilos, extensiones, recursos gráficos y memoria.
CU-023	Abrir un proyecto	Restaura un proyecto guardado, con su señal de origen y el trabajo registrado sobre ella.
CU-003	Cargar señal (con gestión de memoria)	Carga un archivo de señal como nuevo proyecto activo, liberando previamente la memoria del proyecto anterior.
CU-021	Cerrar un proyecto	Cierra el proyecto activo, ofrece guardarlo y libera la memoria asociada a la señal y sus derivados.
CU-020	Orquestador del núcleo (ejecutar análisis pesado)	Recibe, encola y ejecuta fuera del hilo de la interfaz las operaciones de cómputo pesado y publica sus resultados en el estado compartido.
CU-014	Cancelar procesamiento pesado	Interrumpe una tarea en curso, libera su hilo trabajador y descarta el resultado parcial.

El diagrama organiza estos casos de uso en tres agrupamientos que corresponden a tres ciclos de vida distintos. El primero es el ciclo de vida de la aplicación, formado por CU-011 y CU-022. CU-011 deja disponibles los servicios del núcleo mediante inversión de control, de modo que ninguna extensión los instancia por su cuenta, y registra las extensiones sin que sea necesario modificar el núcleo ni las ya existentes. CU-022 es su contraparte: garantiza que al terminar la ejecución no queden hilos, procesos ni contextos gráficos activos.

El segundo agrupamiento corresponde al ciclo de vida del proyecto y lo forman CU-023, CU-003 y CU-021. En el diagrama, CU-021 recibe relaciones de inclusión desde CU-023, CU-003 y CU-022. Esto lo convierte en el punto único de cierre del proyecto: toda operación que reemplace o termine el proyecto activo reutiliza el mismo comportamiento para ofrecer el guardado y liberar la memoria. A su vez, CU-023 incluye a CU-011 y a CU-003, lo que expresa que abrir un proyecto puede implicar iniciar la aplicación y que la restauración de la señal se apoya en el mismo mecanismo de carga. CU-021 también se relaciona con el subsistema Exportación mediante la inclusión de CU-005, del que obtiene la persistencia del proyecto.

El tercer agrupamiento corresponde al ciclo de vida de las tareas de cómputo y lo forman CU-020 y CU-014. CU-020 es el elemento central del subsistema y del modelo completo, ya que recibe las relaciones de inclusión de los casos de uso de Preprocesamiento y Análisis, además de la de CU-003. Su función es desacoplar a las extensiones de la gestión de la concurrencia. Las

extensiones no administran hilos propios, sino que solicitan la ejecución al núcleo. El orquestador encola las solicitudes y las ejecuta en hilos trabajadores, mantiene la interfaz responsiva durante el cómputo y publica los resultados en el estado compartido mediante escritura sincronizada con un único propietario. CU-014 extiende a CU-020 como un comportamiento opcional, activado por el usuario, que propaga la cancelación a la tarea en ejecución y garantiza la liberación de su hilo trabajador, aunque el cómputo no haya terminado.

En conjunto, el subsistema concentra las responsabilidades transversales del sistema: la administración de servicios y extensiones, la gestión de memoria entre proyectos y el control de la ejecución concurrente. Al centralizar estas responsabilidades en el núcleo, los demás subsistemas pueden limitarse a su lógica funcional, y la incorporación de nuevas extensiones se reduce a registrarlas en el núcleo y conectarlas al orquestador.

4.1.6. Subsistema Preprocesamiento

El subsistema **Preprocesamiento** agrupa las operaciones que acondicionan la señal antes de su análisis. Su responsabilidad es transformar la señal y sus segmentaciones sin comprometer la integridad de los datos originales, por lo que concentra requerimientos funcionales de transformación y requerimientos de soporte orientados al encapsulamiento del modelo de datos. El Cuadro 9 resume los casos de uso que lo componen.

Cuadro 9: Casos de uso del subsistema Preprocesamiento

Caso de uso	Nombre	Descripción
CU-004	Generar y modificar trials	Segmenta la señal en ensayos a partir de un canal de estímulo y permite editar o descartar trials ya generados.
CU-018	Ajustar frecuencia de muestreo	Corrige la frecuencia de muestreo de la señal activa y recalcula su eje temporal.
CU-016	Aplicar filtros digitales	Aplica un filtro Butterworth pasa-banda de fase cero y registra el resultado como una señal derivada.
CU-007	Remover artefacto con medidas	Elimina artefactos de estimulación en los trials, mostrando antes las medidas del segmento afectado.
CU-006	Seleccionar trials para análisis	Define el subconjunto de trials que participa en un análisis y deja constancia de cuáles se usaron.

En el diagrama, todos los casos de uso del subsistema, con excepción de CU-018, mantienen una relación de inclusión con el orquestador del núcleo (CU-020). Esto expresa que ninguna transformación se ejecuta en el hilo de la interfaz: el subsistema define qué operación se aplica sobre la señal, y el núcleo determina cómo y cuándo se ejecuta. Así, las extensiones de preprocesamiento contienen solo su lógica de transformación y quedan desacopladas de la gestión de la concurrencia.

La única relación interna del subsistema es la inclusión de CU-004 por parte de CU-018. Como los trials se segmentan sobre el eje temporal de la señal, un cambio en la frecuencia de muestreo deja desactualizadas las segmentaciones existentes, y el sistema ofrece regenerarlas con el mismo mecanismo de corte. Por esta vía, CU-018 llega al orquestador de forma indirecta, a través de CU-004.

Más allá de las relaciones representadas, los casos de uso del subsistema comparten un mismo principio de diseño: los datos originales no se modifican. Cada transformación se registra como un

conjunto derivado de la señal o de los trials, y el acceso al modelo se hace siempre a través de su API pública, sin que ninguna extensión manipule su estructura interna. Cuando un cambio afecta a datos ya representados o analizados, el sistema invalida las cachés dependientes y advierte qué resultados quedaron desactualizados. Con este principio se conserva la trazabilidad entre la señal original y cada una de sus transformaciones, y se reduce el acoplamiento entre las extensiones y el modelo de datos.

En conjunto, el subsistema actúa como la etapa de acondicionamiento previa al análisis. Combina la delegación de la ejecución en el núcleo con la preservación de los datos originales, de modo que el investigador puede probar distintas transformaciones sin perder la señal de partida.

4.1.7. Subsistema Análisis

El subsistema **Análisis** agrupa los cálculos que extraen información de la señal acondicionada, como el acoplamiento entre bandas de frecuencia, la representación tiempo-frecuencia y el promedio de ensayos. Concentra requerimientos funcionales de cálculo y visualización de resultados, junto con requerimientos de rendimiento que acotan el tiempo de los cálculos más costosos. El Cuadro 10 resume los casos de uso que lo componen.

Cuadro 10: Casos de uso del subsistema Análisis

Caso de uso	Nombre	Descripción
CU-001	Análisis PAC de un canal	Calcula el acoplamiento fase-amplitud entre dos bandas de frecuencia de un mismo canal, sobre un trial único o el promedio de trials.
CU-002	Análisis PAC de dos canales / PPC	Calcula el acoplamiento fase-amplitud entre dos canales distintos, o la coherencia de fase entre ellos.
CU-013	Calcular Wavelet	Calcula la transformada wavelet de la señal con normalización y escala configurables, y presenta el escalograma tiempo-frecuencia.
CU-015	Cálculo de promedio de trials	Promedia todos los trials o un subconjunto seleccionado para reducir el ruido de la señal.

En el diagrama, todos los casos de uso concretos del subsistema mantienen una relación de inclusión con el orquestador del núcleo (CU-020). Esto es consecuente con la naturaleza del subsistema, que reúne los cálculos más costosos del sistema. Delegar su ejecución en el núcleo permite que la interfaz siga respondiendo durante el cálculo, que el usuario pueda cancelarlo y que los resultados se publiquen en el estado compartido de forma sincronizada. Cada extensión de análisis queda así limitada a su lógica de cálculo, sin intervenir en la gestión de la concurrencia.

La relación interna del subsistema es una generalización: CU-001 y CU-002 especializan el caso de uso abstracto Ejecutar análisis PAC / PPC, que define el procedimiento común de cálculo del acoplamiento fase-amplitud y sus visualizaciones. Las especializaciones se distinguen por el origen de los datos. CU-001 toma la fase y la amplitud de un mismo canal, y CU-002 las toma de canales distintos y añade la coherencia de fase. Esta estructura permite reutilizar el comportamiento común y agregar nuevas variantes de acoplamiento sin modificar las existentes.

CU-013 y CU-015 no participan en esta jerarquía y se representan como análisis independientes. CU-013 produce la representación tiempo-frecuencia de la señal y tiene un tiempo máximo de cómputo definido. CU-015 produce la serie promediada de los trials y opera sobre el subconjunto seleccionado en el subsistema Preprocesamiento.

En conjunto, el subsistema consume los datos acondicionados por el preprocesamiento y produce los resultados que después se visualizan y exportan. Al delegar la ejecución en el núcleo y organizar los análisis de acoplamiento mediante generalización, el subsistema puede crecer con nuevos análisis sin afectar la estabilidad de la interfaz ni la estructura de los análisis existentes.

4.1.8. Subsistema Exportación

El subsistema **Exportación** agrupa los casos de uso que trasladan el trabajo realizado en Gamma Lab 2.0 al sistema de archivos local. Su responsabilidad es la persistencia del proyecto y la exportación de resultados numéricos y gráficos, por lo que concentra requerimientos funcionales de formato y nomenclatura de archivos, junto con requerimientos de rendimiento que acotan el tiempo de cada exportación. El Cuadro 11 resume los casos de uso que lo componen.

Cuadro 11: Casos de uso del subsistema Exportación

Caso de uso	Nombre	Descripción
CU-005	Guardado y exportación de datos	Guarda el proyecto en formato .glab o exporta los resultados del análisis activo en formatos tabulares (Excel, CSV, JSON) o de imagen rasterizada (JPEG, PNG).
CU-017	Exportación gráfica vectorial (EMF)	Exporta las visualizaciones de los análisis en formato vectorial, conservando sus ejes y escalas para uso en publicaciones.

En el diagrama, el subsistema se conecta con el resto del sistema mediante una única relación: la inclusión de CU-005 por parte de CU-021, Cerrar un proyecto, del subsistema Núcleo y Ciclo de Vida. Esta relación expresa que el guardado del proyecto no es una operación aislada, sino una parte obligatoria del ciclo de vida del proyecto: toda operación que cierre o reemplace el proyecto activo pasa por el mismo mecanismo de persistencia. El archivo de proyecto almacena una referencia a la señal de origen junto con los trials, los análisis y sus parámetros, sin duplicar los datos de la señal. Así, el proyecto se mantiene liviano y puede restaurarse a partir del archivo original.

CU-017 se representa sin relaciones con otros casos de uso, porque es una operación que el usuario invoca directamente sobre una visualización ya generada. Complementa a CU-005 con un formato vectorial que conserva cada elemento gráfico por separado, en lugar de una imagen plana. Los dos casos de uso comparten las mismas convenciones de salida: el nombre sugerido del archivo incluye el nombre de la señal original y los tiempos de exportación están acotados según el tipo de contenido.

En conjunto, el subsistema cierra el flujo de trabajo del sistema, ya que convierte los resultados producidos por los subsistemas Preprocesamiento y Análisis en artefactos persistentes y reutilizables fuera de la aplicación. Al integrarse con el núcleo a través del cierre del proyecto, el sistema asegura que el trabajo del investigador pueda conservarse antes de liberar los recursos de la sesión.

4.2. Vista Lógica

4.2.1. Propósito

La vista lógica describe la descomposición funcional de Gamma Lab 2.0 en clases, interfaces y abstracciones, junto con las responsabilidades de cada elemento y las relaciones estructurales que los vinculan. Esta vista describe los elementos que lo componen, la responsabilidad que asume cada uno de ellos y la forma en que colaboran para satisfacer los requerimientos funcionales.

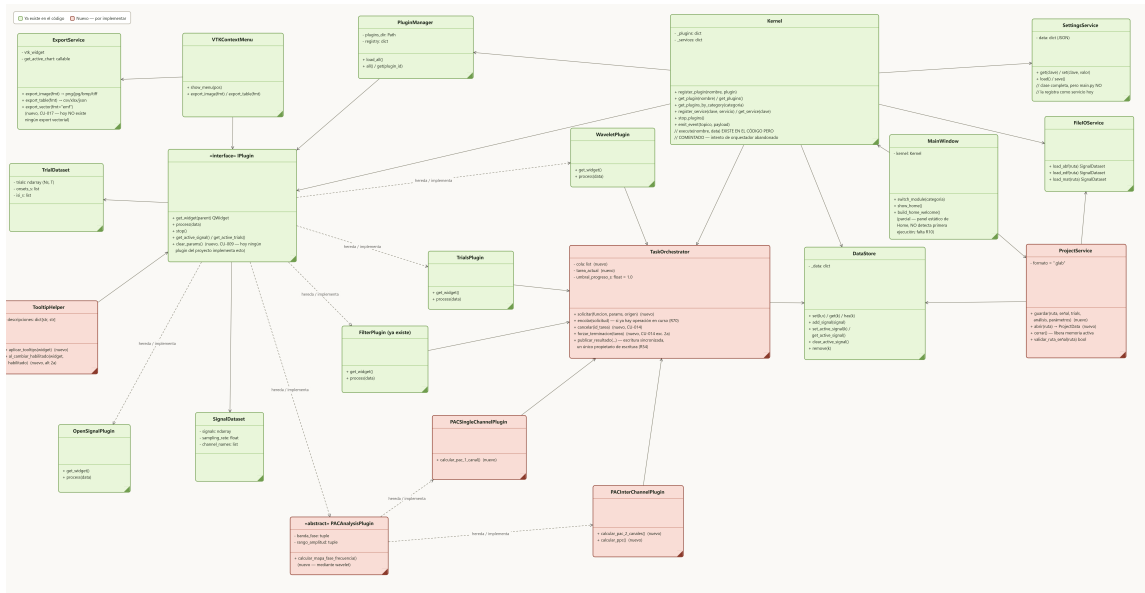
4.2.2. Notación

Dado que Gamma Lab 2.0 evoluciona a partir de un sistema existente, cada elemento del modelo se clasifica según su estado de implementación. Los elementos representados en color verde se encontraban implementados en el código fuente de gammaLab 1.0, mientras que los elementos representados en color rojo están planificados para gammaLab 2.0 y deben construirse para satisfacer casos de uso o requerimientos pendientes. Esta distinción permite que el modelo sirva simultáneamente como descripción del sistema pre-existente y como especificación de la arquitectura objetivo, es decir gammaLab 2.0.

4.2.3. Visión general del modelo

La estructura lógica de Gamma Lab 2.0 responde al estilo arquitectónico de microkernel, en el cual un núcleo mínimo administra un conjunto de extensiones independientes que aportan la funcionalidad del sistema. Los elementos del modelo se organizan en seis agrupaciones. El núcleo administra el ciclo de vida de las extensiones y el registro de servicios. Los servicios del sistema proveen capacidades transversales de configuración, entrada y salida de archivos, exportación, persistencia de proyectos, estado compartido y ejecución de tareas. La capa de presentación agrupa los elementos de interfaz gráfica que coordinan la interacción con el usuario. El contrato de extensión define la interfaz que todo módulo funcional debe cumplir. Las extensiones implementan las operaciones y los análisis sobre las señales. Finalmente, el modelo de datos representa las señales, sus segmentaciones y los proyectos persistidos.

La Figura 3 presenta el diagrama de clases del sistema.



4.2.4. Elementos del núcleo

PluginManager El PluginManager se encuentra implementado y es responsable de descubrir, cargar e instanciar las extensiones disponibles en el directorio de extensiones del sistema. Conserva la ubicación de dicho directorio y un registro de las extensiones descubiertas. Sus operaciones permiten cargar la totalidad de las extensiones (`load_all`) y consultarlas de forma colectiva o por identificador (`all` y `get`). El PluginManager depende de la interfaz IPlugin, de modo que únicamente admite elementos que cumplan dicho contrato.

SettingsService El SettingsService se encuentra implementado y gestiona la configuración persistente de la aplicación. La configuración se mantiene en memoria como un diccionario y se almacena en disco en formato JSON. El servicio ofrece operaciones para consultar y modificar valores de configuración por clave (get y set) y para cargar y almacenar la configuración (load y save).

FileIOService El FileIOService se encuentra implementado y adapta los formatos de archivo soportados al modelo de datos interno del sistema. Sus operaciones `load_abf`, `load_edf` y `load_mat` leen archivos en los formatos ABF, EDF y MAT, respectivamente, y retornan en todos los casos un objeto `SignalDataset`. De esta manera, el resto del sistema permanece independiente de la representación de las señales en disco.

ExportService El ExportService se encuentra implementado y es responsable de exportar visualizaciones y resultados tabulares a formatos externos. Mantiene una referencia al componente de visualización y una función que provee la gráfica activa en cada momento. La operación `export_image` exporta la visualización a formatos de imagen ráster (PNG, JPG, BMP y TIFF) y la operación `export_table` exporta los resultados a formatos tabulares (CSV, XLSX y JSON). Se encuentra planificada la operación `export_vector`, que permitirá la exportación a formato vectorial EMF en respuesta al caso de uso CU-017.

DataStore El DataStore se encuentra implementado y mantiene en memoria el estado compartido de la sesión de trabajo, el cual comprende las señales cargadas, la señal activa, las mediciones y los resultados de los análisis. Constituye la fuente única de verdad del estado de la aplicación. Ofrece operaciones genéricas de almacenamiento, consulta y verificación por clave (`set`, `get` y `has`), operaciones para incorporar señales (`add_signal`), para establecer, consultar y liberar la señal activa (`set_active_signal`, `get_active_signal` y `clear_active_signal`) y para eliminar entradas del estado (`remove`).

TaskOrchestrator El TaskOrchestrator tiene la responsabilidad de coordinar la ejecución de las operaciones de cómputo solicitadas por las extensiones. Su propósito es garantizar que dichas operaciones se ejecuten de forma ordenada, que puedan ser canceladas por el usuario, que se presente retroalimentación de progreso y que sus resultados se publiquen de manera consistente. Para ello mantendrá una cola de solicitudes pendientes, una referencia a la operación en ejecución y un umbral de tiempo, fijado en un segundo, a partir del cual se presentará el indicador de progreso.

La operación `solicitar` constituirá el punto de entrada único mediante el cual una extensión pide la ejecución de un cómputo. La operación `encolar` diferirá una solicitud cuando exista otra operación en curso, en cumplimiento del requerimiento R70. Las operaciones `cancelar` y `forzar_terminacion` permitirán, respectivamente, la cancelación de una tarea a petición del usuario y la terminación forzada de una tarea que no responda a la cancelación, conforme al caso de uso CU-014 y su excepción 2a. Por último, la operación `publicar_resultado` escribirá los resultados en el estado compartido de forma sincronizada. El TaskOrchestrator será el único componente autorizado para escribir en el DataStore, en cumplimiento del requerimiento R54.

ProjectService El ProjectService se encuentra implementado y gestiona la persistencia del espacio de trabajo en archivos de proyecto con extensión `glab`. Cada archivo de proyecto es un manifiesto en formato JSON que registra la señal de origen, las mediciones realizadas, la configuración de la segmentación en ensayos y el estado de los análisis. Durante el guardado, el archivo de señal se copia a la carpeta del proyecto, de modo que el proyecto resulta autocontenido y puede reabrirse en otro equipo. Durante la apertura, el servicio verifica la disponibilidad del archivo de señal referenciado, lo recarga mediante el FileIOService y restaura el estado de la sesión en el DataStore.

El servicio mantiene la carpeta del proyecto, la ruta del archivo de proyecto, el nombre del proyecto, un indicador de cambios pendientes de guardar y una función de notificación que informa

a la interfaz sobre la apertura, el guardado, el cambio de nombre o el cierre del proyecto. Sus operaciones principales permiten guardar el proyecto en su ubicación actual (save), guardarlo en una nueva ubicación (save_as), abrir y restaurar un proyecto existente (open), y cerrar el proyecto activo liberando su estado (close). Adicionalmente, ofrece operaciones para asignar el nombre del proyecto (set_project_name), registrar la existencia de cambios pendientes (mark_dirty), verificar si existe un proyecto abierto (is_open) y generar un nombre predeterminado a partir del nombre de la señal (default_project_name). El ProjectService depende del FileIOService, del DataStore y del Kernel, y produce y consume objetos de tipo ProjectData.

4.2.6. Elementos de presentación

MainWindow La MainWindow se encuentra parcialmente implementada y constituye el contenedor principal de la interfaz gráfica. Es responsable de la navegación entre los módulos funcionales, de la presentación de la pantalla de inicio y de las operaciones de proyecto disponibles en el menú Archivo. Mantiene referencias al Kernel y al ProjectService. La operación switch_module presenta las extensiones correspondientes a la categoría seleccionada por el usuario, la operación (show_home) presenta la pantalla de inicio y la operación (build_home_welcome) construye el panel de bienvenida. Esta última se encuentra pendiente de incorporar la detección de la primera ejecución de la aplicación, requerida por el requerimiento R10.

VTKContextMenu El VTKContextMenu se encuentra implementado y provee el menú contextual asociado a las visualizaciones. Sus operaciones permiten presentar el menú en la posición indicada por el usuario (show_menu) y solicitar la exportación de la visualización como imagen o como tabla (export_image y export_table). Este elemento delega la exportación en el ExportService y opera sobre los componentes visuales provistos por las extensiones.

TooltipHelper El TooltipHelper se encuentra planificado y tendrá la responsabilidad de proveer ayuda contextual sobre los controles de la interfaz. Mantendrá un diccionario de descripciones asociadas a cada control. La operación (aplicar_tooltips) asignará dichas descripciones a los controles de un componente visual y la operación (al_cambiar_habilitado) actualizará la descripción de un control cuando cambie su disponibilidad, de modo que el usuario conozca el motivo por el cual un control se encuentra deshabilitado.

4.2.7. Contrato de extensión

IPlugin IPlugin es la interfaz que define el contrato que toda extensión debe cumplir para integrarse al sistema, y constituye el mecanismo principal de extensibilidad de la arquitectura. Se encuentra implementada y establece las siguientes operaciones: get_widget, que provee la interfaz gráfica de la extensión; process, que ejecuta el procesamiento propio de la extensión; stop, que detiene la extensión y libera sus recursos; y get_active_signal y get_active_trials, que proporcionan acceso a los datos de trabajo. IPlugin depende de las clases SignalDataset y TrialDataset.

Se encuentra planificada la incorporación al contrato de la operación clear_params, cuya finalidad es restablecer los parámetros de una extensión a sus valores predeterminados, conforme al caso de uso CU-009. Actualmente esta operación se encuentra implementada de forma individual en las extensiones FilterPlugin, TrialsPlugin y ArtifactRemovePlugin, pero aún no forma parte de la interfaz común.

4.2.8. Extensiones

Todas las extensiones del sistema realizan la interfaz IPlugin. La Tabla 12 resume las extensiones representadas en el modelo, su estado de implementación y su responsabilidad.

Cuadro 12: Extensiones representadas en la vista lógica.

Elemento	Responsabilidad
OpenSignalPlugin	Selección y carga de señales desde archivo.
FilterPlugin	Filtrado en frecuencia de la señal.
WaveletPlugin	Análisis en tiempo y frecuencia mediante la transformada wavelet.
TrialsPlugin	Segmentación de la señal en ensayos.
PACAnalysisPlugin	Comportamiento común del análisis de acoplamiento entre fase y amplitud (PAC).
PACSingleChannelPlugin	Cálculo del acoplamiento entre fase y amplitud dentro de un mismo canal.
PACInterChannelPlugin	Cálculo del acoplamiento entre fase y amplitud y de la consistencia de fase por pares (PPC) entre dos canales.

PACAnalysisPlugin es una clase abstracta que concentra los parámetros y el procedimiento comunes a todo análisis de acoplamiento entre fase y amplitud. Sus atributos definen la banda de frecuencia de la fase y el rango de frecuencia de la amplitud, y su operación `calcular_mapa_fase_frecuencia` obtiene el mapa de fase y frecuencia mediante la transformada wavelet. Las clases PACSingleChannelPlugin y PACInterChannelPlugin especializan este comportamiento. La primera incorpora la operación `calcular_pac_1_canal`, que calcula el acoplamiento dentro de un mismo canal, y la segunda incorpora las operaciones `calcular_pac_2_canales` y `calcular_ppc`, que calculan el acoplamiento y la consistencia de fase entre dos canales.

4.2.9. Modelo de datos

SignalDataset SignalDataset se encuentra implementada y representa una señal electrofisiológica multicanal. Sus atributos contienen la matriz de muestras de la señal, la frecuencia de muestreo expresada en hercios y los identificadores de los canales.

TrialDataset TrialDataset se encuentra implementada y representa una señal segmentada en ensayos. Sus atributos contienen la matriz de ensayos, cuyas dimensiones corresponden al número de muestras por ensayo y al número de ensayos, los instantes de inicio de cada ensayo expresados en segundos y los intervalos entre estímulos expresados en segundos.

ProjectData ProjectData es una estructura de datos implementada que representa en memoria el contenido de un archivo de proyecto. En la implementación actual se materializa como un diccionario serializado en formato JSON, producido por la operación `save` y retornado por la operación `open` del ProjectService. Sus atributos registran la versión del esquema del formato de proyecto, el nombre del proyecto, la fecha y hora del último guardado, la información de la señal de origen (nombre de archivo, formato, nombre y frecuencia de muestreo), las mediciones

realizadas, la configuración de la segmentación en ensayos y los parámetros y el estado de las extensiones de análisis. La versión del esquema permite detectar incompatibilidades entre versiones del formato al momento de abrir un proyecto.

4.2.10. Relaciones estructurales

La Tabla 13 resume las relaciones entre los elementos del modelo lógico.

Cuadro 13: Relaciones estructurales de la vista lógica.

Origen	Tipo	Destino	Descripción
Kernel	Asociación	PluginManager	Delega la carga de extensiones.
Kernel	Asociación	SettingsService, FileIOService, DataStore, TaskOrchestrator	Registra y expone los servicios del sistema.
Kernel, PluginManager	Dependencia	IPlugin	Administran las extensiones a través de su contrato.
MainWindow	Asociación	Kernel, ProjectService	Accede a las extensiones, a los servicios y a las operaciones de proyecto.
ProjectService	Dependencia	FileIOService, DataStore	Recarga la señal y restaura el estado de la sesión.
ProjectService	Dependencia	ProjectData	Produce y consume el contenido del proyecto.
VTKContextMenu	Dependencia	ExportService	Delega la exportación.
VTKContextMenu, TooltipHelper	Dependencia	IPlugin	Operan sobre los componentes visuales de las extensiones.
IPlugin	Dependencia	SignalDataset, TrialDataset	Manipula el modelo de datos.
Extensiones concretas y PACAnalysisPlugin	Realización	IPlugin	Implementan el contrato de extensión.
PACSingleChannelPlugin, PACInterChannelPlugin	Generalización	PACAnalysisPlugin	Especializan el análisis de acoplamiento.
Extensiones de análisis	Dependencia	TaskOrchestrator	Solicitan la ejecución de operaciones de cómputo.
TaskOrchestrator	Asociación	DataStore	Publica los resultados en el estado compartido.

4.2.11. Justificación de las decisiones

La separación entre el núcleo y las extensiones mediante la interfaz IPlugin favorece la extensibilidad del sistema, ya que permite incorporar nuevos análisis, como los de acoplamiento entre fase y amplitud, sin modificar el código existente, en conformidad con el principio de abierto y cerrado.

La asignación exclusiva de la escritura sobre el estado compartido al TaskOrchestrator garantiza la consistencia del estado de la aplicación, al eliminar las condiciones de carrera entre operaciones concurrentes, conforme al requerimiento R54.

La centralización de la ejecución de los cálculos en el TaskOrchestrator mejora la capacidad de respuesta del sistema y el control que ejerce el usuario sobre él, puesto que habilita la ejecución ordenada de las operaciones (R70), su cancelación (CU-014) y la presentación de progreso, sin que cada extensión deba gestionar sus propios hilos de ejecución.

La concentración de la lectura de archivos en el FileIOService asegura la independencia del sistema respecto de los formatos de señal, y confina a un único elemento el impacto de incorporar nuevos formatos.

La persistencia de proyectos mediante el ProjectService y la estructura ProjectData produce proyectos autocontenidos y versionados. La inclusión de una copia de la señal de origen permite reabrir el trabajo en otro equipo, y la versión del esquema permite detectar incompatibilidades entre versiones del formato.

Por último, la jerarquía encabezada por PACAnalysisPlugin promueve la reutilización, al evitar la duplicación del cálculo del mapa de fase y frecuencia entre las variantes de análisis de uno y de dos canales.

4.2.12. Trazabilidad con los requerimientos

La Tabla 14 relaciona los elementos y operaciones del modelo lógico con los casos de uso y requerimientos que los originan.

Cuadro 14: Trazabilidad entre elementos de la vista lógica y requerimientos.

Elemento u operación	Caso de uso o requerimiento
IPlugin, operación clear_params	CU-009
TaskOrchestrator, operación cancelar	CU-014
TaskOrchestrator, operación forzar_terminacion	CU-014, excepción 2a
ExportService, operación export_vector	CU-017
ProjectService, operaciones save, save_as y close	CU-021, R07
ProjectService, operación open, y ProjectData	CU-023
TaskOrchestrator, operación publicar_resultado	R54
TaskOrchestrator, operación encolar	R70
MainWindow, operación build_home_welcome	R10
PACSingleChannelPlugin y PACInterChannelPlugin	CU-001 y CU-002

4.3. Vista de Procesos

La vista de procesos describe el comportamiento dinámico del sistema completo como un único flujo maestro de sesión, construido a partir de las cadenas de precondition declaradas por las propias 23 fichas (p. ej. CU-006 exige trials generados por CU-004, que a su vez exige una señal cargada por CU-003) — no es una fusión literal de los 23 flujos detallados, que sería ilegible.

El flujo de la Figura 4 se organiza en las siguientes etapas:

1. **Inicio.** El sistema registra los servicios del núcleo y descubre los plugins vía YAML (CU-011); el registro del servicio de tareas en este paso es nuevo, el resto ya existe en `main.py`.
2. **Apertura o carga.** Según si el investigador abre un `.glab` (CU-023, nuevo) o carga una señal desde Home vacío (CU-003, ya existe vía `FileIOService`), la señal activa termina publicándose en `DataStore`.

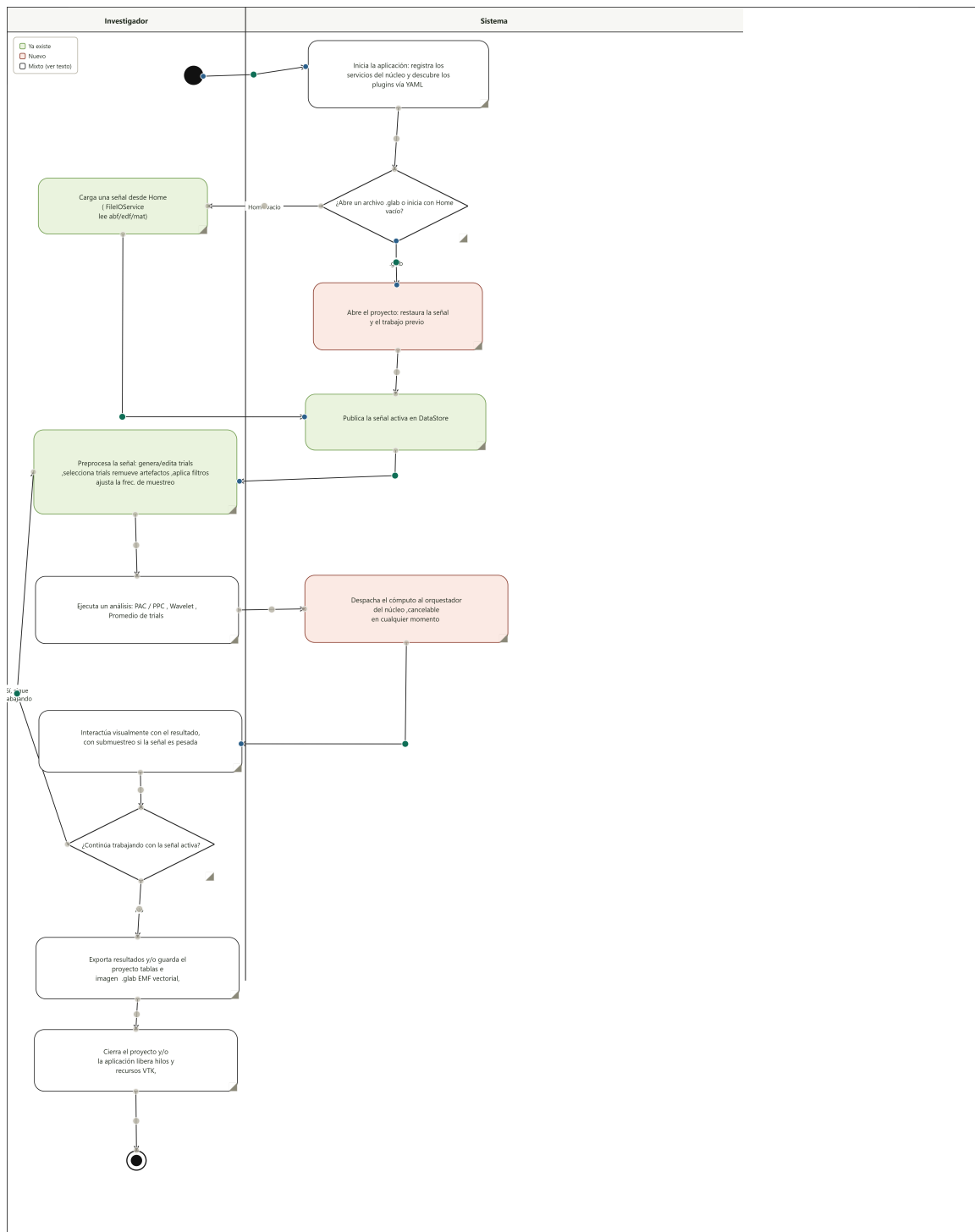


Figura 4: Flujo maestro de sesión de Gamma Lab 2.0, con carriles Investigador/Sistema. Verde = ya existe, rojo = nuevo, blanco = mixto.

3. **Preprocesamiento.** Generación/edición de trials, selección de trials, remoción de artefactos, filtrado y ajuste de frecuencia de muestreo (CU-004, CU-006, CU-007, CU-016, CU-018) — los cuatro plugins ya existen en el código.
4. **Análisis.** PAC/PPC (CU-001, CU-002, nuevos), Wavelet (CU-013), promedio de trials (CU-015).
5. **Orquestación.** Todo cómputo pesado se despacha al orquestador del núcleo (CU-020, nuevo), cancelable en cualquier momento (CU-014, nuevo).
6. **Visualización.** Interacción visual con el resultado, con submuestreo si la señal es pesada (CU-012 parcial + CU-019, nuevo).
7. **Iteración o cierre.** Si el investigador continúa trabajando, el flujo retorna al preprocesamiento; de lo contrario, exporta y/o guarda el proyecto (CU-005 parcial, CU-017 nuevo) y cierra el proyecto y/o la aplicación (CU-021 nuevo, CU-022 parcial).

A diferencia de Gamma Lab 1.0, los casos de uso de navegación (CU-008), limpieza de formulario (CU-009, 0 % implementado) y ayuda contextual (CU-010, cobertura parcial en solo 2 de más de 15 plugins) son transversales a toda la sesión: no ocupan un lugar fijo en la secuencia y se documentan como notas adjuntas al diagrama en lugar de forzarlas dentro del flujo principal.

4.4. Vista Física

La vista física describe el entorno de despliegue local de Gamma Lab 2.0. A diferencia de Gamma Lab 1.0, aquí se documenta explícitamente la superficie completa de entrada/salida del sistema, incluyendo los artefactos que aún no existen.

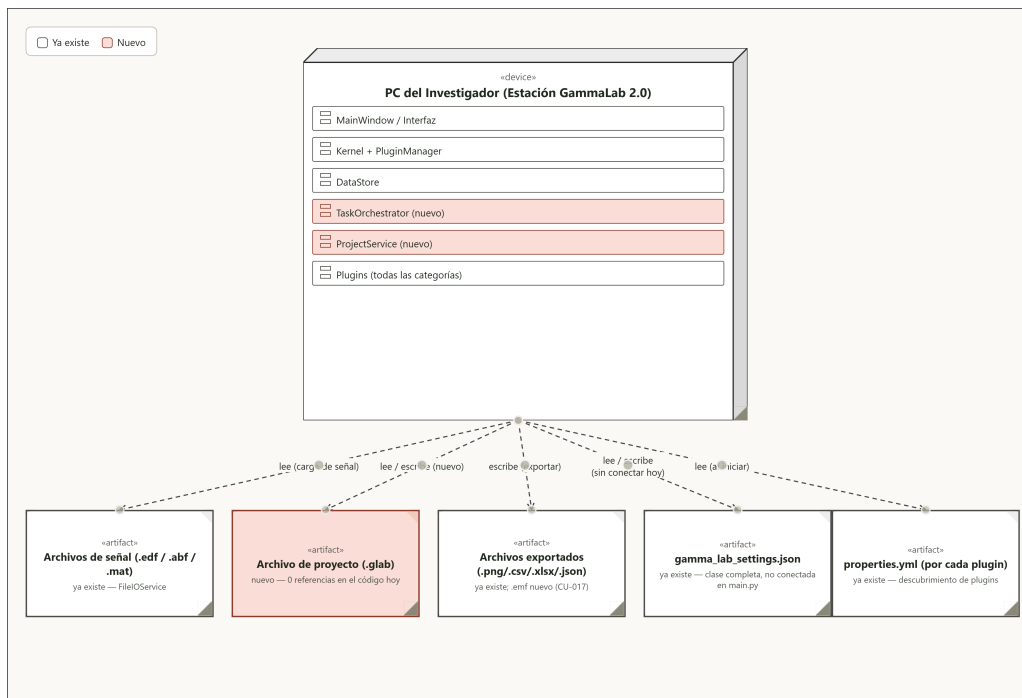


Figura 5: Vista física de Gamma Lab 2.0: estación única de escritorio y su superficie de E/S.

La Figura 5 muestra el despliegue típico:

- Un único nodo *PC del Investigador*: no existe arquitectura cliente-servidor en ninguna de las 23 fichas.
- **Archivos de señal (.edf/.abf/.mat):** ya existe, leídos por FileIOService.
- **Archivo de proyecto (.glab):** nuevo — cero referencias en el código actual.

- **Archivos exportados** (.png/.csv/.xlsx/.json): ya existe vía `ExportService`; el formato vectorial `.emf` (CU-017) es nuevo.
- `gamma_lab_settings.json`: la clase `SettingsService` ya existe, pero **no** está conectada en el arranque (`main.py`) hoy.
- `properties.yml` (por cada plugin): ya existe, sostiene el descubrimiento automático de plugins.

5. Atributos de calidad

5.1. Escenarios de calidad

5.1.1. Escenario de Calidad 1: Integración de un Plugin Externo

Cuadro 15: Escenario de calidad 1 — Integración de un plugin externo

Aspecto	Descripción
Atributo de calidad	Soporte
Requisito	El sistema debe permitir integrar plugins externos sin recompilar el núcleo y sin afectar módulos existentes.
Fuente del estímulo	Desarrollador externo
Estímulo	Se agrega un nuevo plugin a la carpeta de Plugins siguiendo el formato definido por el Kernel.
Artefacto	Módulo core del sistema (Kernel) y módulo de plugins.
Ambiente	Sistema en operación normal con múltiples plugins ya registrados.
Respuesta	El sistema detecta, valida y registra el plugin, exponiéndolo en la interfaz de usuario.
Medida de Respuesta	Plugin visible en la UI en menos de 1 minuto. Ningún plugin existente debe fallar tras la carga.
Trazabilidad	NFR-S-05 (R77–R78)

5.1.2. Escenario de Calidad 2: Carga de Señal de Gran Tamaño

Cuadro 16: Escenario de calidad 2 — Carga de señal de gran tamaño

Aspecto	Descripción
Atributo de calidad	Rendimiento
Requisito	El sistema debe gestionar señales de gran tamaño sin bloqueos ni degradación significativa del rendimiento.
Fuente del estímulo	Usuario final
Estímulo	Se carga un archivo ABF/EDF de gran tamaño (ej. 1–2 GB, múltiples canales).
Artefacto	FileIOService, DatasetService (DataStore), servicio de tareas del núcleo, interfaz de usuario.
Ambiente	Computador con carga moderada y otros procesos del sistema activos.
Respuesta	La señal se carga correctamente, sin duplicación innecesaria de datos y sin bloqueos de la interfaz; la lectura corre en el servicio de tareas, fuera del hilo de la UI.
Medida de Respuesta	Visualización inicial en menos de 8 segundos para este caso de estrés (el umbral base de carga es ≤ 2 s para el archivo típico de referencia). Uso de CPU menor a 80 %. Sin cierres inesperados ni latencias mayores a 5 segundos.
Trazabilidad	NFR-P-03 (R48), NFR-P-14 (R72), NFR-P-10 (R56)

5.1.3. Escenario de Calidad 3: Manejo de Errores en Tiempo de Ejecución

Cuadro 17: Escenario de calidad 3 — Manejo de errores en tiempo de ejecución

Aspecto	Descripción
Atributo de calidad	Confiabilidad
Requisito	El sistema debe manejar excepciones en los plugins sin comprometer la estabilidad del sistema.
Fuente del estímulo	Sistema o entorno
Estímulo	Un plugin genera una excepción interna durante un análisis (FFT, PSD, Wavelet, ERP, PAC, etc.).
Artefacto	Módulo de plugins, Kernel, servicios asociados a cada plugin.
Ambiente	Análisis en curso con señales cargadas y otros plugins activos.
Respuesta	El sistema captura la excepción, informa al usuario y evita un cierre inesperado de la aplicación, manteniendo el Kernel operando de manera adecuada.
Medida de Respuesta	0 % de pérdida de datos. Continuidad del sistema garantizada. Mensaje de error mostrado en menos de 2 segundos.
Trazabilidad	R47

5.1.4. Escenario de Calidad 4: Exportación de Datos o Gráficas

Cuadro 18: Escenario de calidad 4 — Exportación de datos o gráficas

Aspecto	Descripción
Atributo de calidad	Confiabilidad
Requisito	El sistema debe exportar gráficos y datos sin errores, incluso si se presentan fallas externas por disco, permisos, memoria u otros motivos.
Fuente del estímulo	Usuario final
Estímulo	El usuario exporta una tabla, medición o gráfico en PNG/CSV/JSON.
Artefacto	ExportService (registrado en el núcleo por inversión de control), interfaz de usuario, sistema de archivos.
Ambiente	Sistema operando con múltiples análisis realizados previamente sin guardar.
Respuesta	El archivo se genera correctamente o, en caso de error, se notifica al usuario sin afectar el estado del sistema.
Medida de Respuesta	Tiempo de exportación menor a 3 segundos. 0 % de pérdida de datos. Notificación clara ante error.
Trazabilidad	NFR-P-07 (R51), NFR-P-12 (R57), NFR-S-03 (R70)

5.1.5. Escenario de Calidad 5: Uso del Sistema sin Experiencia Técnica

Cuadro 19: Escenario de calidad 5 — Operación por un usuario sin experiencia técnica

Aspecto	Descripción
Atributo de calidad	Usabilidad
Requisito	El sistema debe ser utilizable por un usuario con experiencia en neurociencias o análisis de señales, pero sin conocimientos profundos del software.
Fuente del estímulo	Usuario final
Estímulo	El usuario utiliza un plugin complejo como Wavelet, PSD o PAC por primera vez.
Artefacto	Interfaz de usuario, ayudas contextuales, parámetros preconfigurados.
Ambiente	Uso sin acompañamiento técnico.
Respuesta	El usuario puede completar el análisis apoyándose en la interfaz, con guía clara y sin errores operativos.
Medida de Respuesta	Análisis realizado en menos de 5 minutos. Bajo número de consultas al manual de usuario.
Trazabilidad	NFR-U-07 (R42), NFR-U-08 (R43), NFR-U-05 (R10), NFR-U-09 (R44)

5.1.6. Escenario de Calidad 6: Rendimiento con Múltiples Señales Cargadas

Cuadro 20: Escenario de calidad 6 — Rendimiento con múltiples señales cargadas

Aspecto	Descripción
Atributo de calidad	Rendimiento
Requisito	El sistema debe mantener tiempos de respuesta estables aun cuando varias señales se encuentren cargadas simultáneamente dentro de la misma ProjectSession.
Fuente del estímulo	Usuario final
Estímulo	El usuario carga entre 2 y 4 señales de gran tamaño y ejecuta diferentes análisis en cada una.
Artefacto	DataStore, gestor de vistas, motor de análisis.
Ambiente	Sistema operando con alto volumen de datos en memoria.
Respuesta	El sistema reutiliza datos sin duplicarlos, mantiene tiempos de acceso aceptables y evita bloqueos en la interfaz.
Medida de Respuesta	Cambio entre señales cargadas en menos de 1 segundo. Sin bloqueos ni cierres inesperados. Uso de memoria dentro del 70–75 %.
Trazabilidad	NFR-P-06 (R51), NFR-P-01/P-08 (R11, R53)

5.1.7. Escenario de Calidad 7: Resiliencia ante Archivos Corruptos o Incompatibles

Cuadro 21: Escenario de calidad 7 — Resiliencia ante archivos corruptos o incompatibles

Aspecto	Descripción
Atributo de calidad	Confiabilidad
Requisito	El sistema debe detectar archivos corruptos, incompletos o incompatibles sin comprometer la estabilidad del programa.
Fuente del estímulo	Usuario final o sistema externo
Estímulo	El usuario intenta cargar un archivo ABF/EDF/MAT con estructura inválida o con metadatos corruptos.
Artefacto	FileIOService, Kernel, sistema de alertas.
Ambiente	Sistema en operación normal con señales previamente cargadas.
Respuesta	El sistema captura el error, notifica al usuario, cancela la operación de carga y mantiene intacta la sesión actual.
Medida de Respuesta	Mensaje de error mostrado en menos de 2 segundos. 0 % de pérdida de datos o señales cargadas previamente. El sistema continúa operativo sin fallos.
Trazabilidad	FR-IO-04

5.1.8. Escenario de Calidad 8: Consistencia entre Datos Ingresados y Gráficas Generadas

Cuadro 22: Escenario de calidad 8 — Consistencia entre datos ingresados y gráficas generadas

Aspecto	Descripción
Atributo de calidad	Confiabilidad y Usabilidad
Requisito	El sistema debe garantizar que las gráficas generadas correspondan fielmente a los datos y parámetros seleccionados por el usuario.
Fuente del estímulo	Usuario final
Estímulo	El usuario modifica parámetros como canal, ventana temporal, trials seleccionados o rango de frecuencias.
Artefacto	Interfaz de usuario, motor de visualización, servicios de análisis.
Ambiente	Sistema ejecutando análisis consecutivos sobre diferentes configuraciones.
Respuesta	La gráfica generada refleja exactamente los datos filtrados o seleccionados, sin desajustes ni inconsistencias con los parámetros ingresados.
Medida de Respuesta	Actualización gráfica en menos de 3 segundos. 0 diferencias entre los datos mostrados y los procesados. Consistencia confirmada entre ejecuciones sucesivas.
Trazabilidad	NFR-R-04 (R73), NFR-U-02 (R03)

5.2. Reglas de compatibilidad y versionado

- **Versionado de la aplicación.** GammaLab 2.0 adopta versionado semántico MAJOR. MINOR. PATCH: MAJOR rompe `.glab` o el contrato `IPlugin`; MINOR agrega capacidades retrocompatibles; PATCH corrige sin alterar contratos públicos. `main.py` expone esta versión en una constante única, visible en un diálogo “Acerca de” y embebida como metadato en cada archivo `.glab` generado.
- **Contrato de plugins (IPlugin).** Cambios aditivos (nuevos métodos opcionales, nuevos campos con valor por defecto) no rompen compatibilidad. Cambios que alteren la firma de `initialize`, `start`, `stop`, `process` o `get_widget` elevan la versión MAJOR del contrato.
- **Metadatos `properties.yml`.** Los campos requeridos (`id`, `name`, `category`, `subcategory`, `version`, `icon`, `logic_class`, `ui_class`) deben existir siempre – exigido por `REQUIRED_KEYS` en `loader.py` – y añadir campos opcionales es compatible. El campo `version` declara el MAJOR de `IPlugin` contra el que se escribió el plugin; `discover()` rechaza el plugin si no coincide con el MAJOR del núcleo en ejecución.
- **Formato de proyecto `.glab`.** `ProjectService` persiste el espacio de trabajo – señal activa, trials y análisis – en archivos `.glab`. Cada `.glab` embebe un `schema_version` propio: un incremento MINOR se mantiene legible por versiones futuras de `ProjectService`, y un incremento MAJOR rechaza la apertura con un mensaje explícito al usuario en vez de migrar implícitamente.
- **Dependencias externas.** `requirements.txt` fija cada librería con versión exacta (`==`), sin rangos abiertos (Tabla 23). Actualizar VTK, SciPy, PyWavelets o NumPy exige re-ejecutar completas las suites `plugins_test` y `performance_test` antes de aceptar el cambio.
- **Formatos de señal externos (EDF/ABF/BDF/MAT).** La compatibilidad se ancla a la versión de la librería lectora (`pyabf`, `pyedflib`), no al formato en sí. Un archivo no

interpretable por esa versión se reporta como incompatible, sin remuestreo ni heurísticas de recuperación implícitas: un EDF con canales a distinta frecuencia de muestreo falla con un mensaje claro, en vez de intentar corregirse automáticamente.

- **Estabilidad de módulos internos.** Todo renombramiento o reubicación de un módulo público de `core/` actualiza en el mismo cambio a todos sus consumidores, tests incluidos, y un flujo de CI corre `pytest` en cada *push/pull request*. Motivo: un renombramiento previo de `fileio.py` a `fileio_service.py` (con `TrialDataset` movido a `core/model/`) rompió silenciosamente los 8 archivos de `plugins_test` al no actualizarse en el mismo cambio.

Cuadro 23: Versiones de dependencias externas ancladas en `requirements.txt`

Librería	Versión	Rol
numpy	2.3.4	Cálculo numérico vectorizado
scipy	1.16.2	Filtros, PSD (Welch)
PyWavelets	1.9.0	Transformada Wavelet Continua
vtk	9.5.2	Renderizado científico 2D/3D
PyQt5 / PyQt5-Qt5	5.15.11 / 5.15.2	Interfaz gráfica
pyabf	2.3.8	Lectura de archivos .abf
pyEDFlib	0.1.42	Lectura de archivos .edf
h5py	3.15.1	Soporte de formatos basados en HDF5
pytest	8.4.2	Framework de pruebas

5.3. Métricas y plan de verificación

La Tabla 24 formaliza la “Medida de Respuesta” de cada escenario de calidad (sección 5.1) en una métrica verificable, junto con su método de medición y el criterio de aceptación correspondiente.

Cuadro 24: Métricas, método de verificación y criterio de aceptación por atributo de calidad

Atributo	Métrica	Método	Criterio de aceptación
Extensibilidad (Esc. 1)	Tiempo de aparición de un plugin nuevo; plugins existentes que fallan	Agregar un plugin válido y correr <code>plugins_test</code>	<1 min; 0 plugins fallan (R77–R78)
Rendimiento – carga grande (Esc. 2)	Tiempo a primer render; uso de CPU	<code>performance_test</code> ; medido: 78.4 ms (1.8 M muestras)	≤8 s estrés, ≤2 s típico; CPU <80 % (R48/R72/R56)
Confiabilidad – errores en ejecución (Esc. 3)	% pérdida de datos; tiempo del mensaje	Inyección de fallos dentro de <code>process()</code>	0 % pérdida; mensaje <2 s; sistema operativo
Confiabilidad – exportación (Esc. 4)	Tiempo de exportación; error silencioso	<code>test_export.py</code>	<3 s; 0 % pérdida; notificación clara (R51/R57/R70)
Usabilidad (Esc. 5)	Tiempo de análisis sin acompañamiento	Prueba de usabilidad con usuarios reales	<5 min; bajas consultas al manual (R42–R44)

Continúa en la siguiente página

Atributo	Métrica	Método	Criterio de aceptación
Rendimiento – múltiples señales (Esc. 6)	Tiempo de cambio; % de memoria	Benchmark de DataStore	<1 s cambio; memoria 70–75 % (R11, R51, R53)
Confiabilidad – archivos corruptos (Esc. 7)	Tiempo del error; continuidad de sesión	<code>test_fileio_abf/edf.py</code>	Mensaje <2 s; 0 % pérdida previa (FR-IO-04)
Confiabilidad y Usabilidad (Esc. 8)	Tiempo de actualización; diferencias	Revisión de consistencia entrada-salida	<3 s; 0 diferencias (R73, R03)
Mantenibilidad	% de cobertura automática	<code>pytest --cov sobre core/plugins</code>	≥80 % de cobertura
Rendimiento – no regresión vs. 1.0	Factor vs. línea base	<code>compare_report.py</code>	Nada empeora; render sin vectorizar mejora ≥10×
Portabilidad	Éxito de arranque del <code>.exe</code>	Instalación en Windows limpio sin Python	Arranque exitoso, sin configuración manual
Robustez – cancelación (UC-02)	Liberación del hilo tras cancelar	Prueba forzada sobre <code>TaskOrchestrator</code>	Liberación garantizada (R46, R70, R74)

5.4. Plan de evidencia

1. **Fixture ABF real y referencias MATLAB.** `test/data/17308005.abf` más los 8 CSV de referencia en `plugins_test/` sustentan la corrección numérica exigida por Extensibilidad (Esc. 1) y por las métricas de Mantenibilidad.
2. **Suite de benchmarks versionada.** `performance_test/`, con resultados persistidos por etiqueta en `perf_results.json` (p.ej. `gammalab1_baseline_2026-08-16`), es la evidencia reproducible de Rendimiento y de la comparación de no regresión frente a 1.0.
3. **Auditorías documentales.** Los informes `reporte_tests.tex` e `informe_rendimiento.tex` (`docs/tests documents/`) registran el estado de la suite de pruebas y los hallazgos de rendimiento.
4. **Pruebas de usabilidad con usuarios reales.** `docs/pruebas-usabilidad/` sustenta el criterio de aceptación de Usabilidad (Esc. 5).
5. **Prueba de instalación limpia.** Arranque del ejecutable generado por PyInstaller en un Windows 10/11 sin Python previamente instalado, como evidencia de Portabilidad.
6. **Registro de cancelación forzada.** La prueba de cancelación forzada sobre el `TaskOrchestrator` bajo carga es la evidencia de Robustez (UC-02).

5.5. Conclusión

Los escenarios de calidad, las reglas de compatibilidad y la tabla de métricas convierten los atributos de calidad de la sección 3.2 en criterios operativos y verificables, no en aspiraciones genéricas. El plan de evidencia de la sección 6.4 sostiene estos criterios con artefactos reproducibles: fixtures reales validados contra una referencia de MATLAB, benchmarks versionados por etiqueta, auditorías documentales, y pruebas de usabilidad y de instalación. Junto con las reglas

de versionado de la sección 6.2, esta estructura garantiza que la arquitectura de GammaLab 2.0 evalúe, documente y demuestre sus atributos de calidad de forma reproducible en cada nueva versión del sistema.

6. Anexos

6.1. Anexo A — Relación con el SRS

En particular:

-

6.2. Anexo B — Relación con el SDD

Este anexo resume dicha relación:

-

6.3. Anexo C — Trazabilidad global de artefactos

6.4. Anexo D — Diagramas y tablas de referencia

7. Referencias

Referencias

- [1] Python Software Foundation. *Python 3 — The Python Standard Library*. Disponible en: <https://docs.python.org/3/>.
- [2] Riverbank Computing. *PyQt5 Reference Guide*. Disponible en: <https://www.riverbankcomputing.com/static/Docs/PyQt5/>.
- [3] The Qt Company. *Qt 5 — Style Sheets Reference*. Disponible en: <https://doc.qt.io/qt-5/stylesheet-syntax.html>.
- [4] Kitware, Inc. *VTK — Visualization Toolkit Documentation*. Disponible en: <https://vtk.org/> y <https://docs.vtk.org/>.
- [5] Kitware, Inc. *vtkChartXY Class Reference*. Disponible en: <https://docs.vtk.org/en/latest/api/python/vtkmodules.vtkChartsCore.html#vtkmodules.vtkChartsCore.vtkChartXY>.
- [6] NumPy Developers. *NumPy Documentation*. Disponible en: <https://numpy.org/doc/>.
- [7] Scott W. Harden. *pyABF — Read and Analyze ABF Files with Python*. Disponible en: <https://pyabf.readthedocs.io/>.
- [8] Holger Nahrstaedt et al. *pyEDFlib — Python library to read/write EDF+/BDF+ files*. Disponible en: <https://pyedflib.readthedocs.io/>.

- [9] Oren Ben-Kiki, Clark Evans, Ingy döt Net. *YAML Ain't Markup Language (YAML) Specification*. Disponible en: <https://yaml.org/spec/>.
- [10] P. B. Kruchten. *The 4+1 View Model of Architecture*. IEEE Software, vol. 12, no. 6, pp. 42–50, 1995. DOI: <https://doi.org/10.1109/52.469759>.
- [11] Len Bass, Paul Clements, Rick Kazman. *Software Architecture in Practice* (3rd ed.). Addison–Wesley, 2012. ISBN 978-0-321-81573-6.
- [12] Paul Clements, Felix Bachmann, Len Bass, David Garlan, James Ivers, Reed Little, Robert Nord, Judith Stafford. *Documenting Software Architectures: Views and Beyond* (2nd ed.). Addison–Wesley, 2010. ISBN 978-0-321-55268-6.
- [13] Rick Kazman, Mark Klein, Paul Clements. *ATAM: Method for Architecture Evaluation*. CMU/SEI-2000-TR-004, Software Engineering Institute, 2000.
- [14] Bob Kemp, Alpo Värri, Agostinho Rosa, Kim D. Nielsen, John Gade. *A simple format for exchange of digitized polygraphic recordings*. Electroencephalography and Clinical Neurophysiology, 82(5), 1992, pp. 391–393. DOI: [https://doi.org/10.1016/0013-4694\(92\)90009-7](https://doi.org/10.1016/0013-4694(92)90009-7).
- [15] P. L. Nunez, R. Srinivasan. *Electric Fields of the Brain: The Neurophysics of EEG* (2nd ed.). Oxford University Press, 2006.
- [16] Equipo Gamma Lab. *Software Architecture Document (SAD)*, v1.0. Este documento.
- [17] Object Management Group. *OMG Unified Modeling Language (OMG UML), Version 2.5.1*. OMG Doc. formal/17-12-05, dic. 2017. Disponible en: <https://www.omg.org/spec/UML/2.5.1/>.

Nota: Las URLs señaladas se incluyen para consulta rápida. En publicación impresa, usar DOI/ISBN cuando aplique.